COLLABORATIVE FILTERING FÖR ATT VÄLJA SPELNIVÅER

Examensarbete Systemarkitekturutbildningen

Fredrik Dahlberg
Mathias Söderqvist

HT 2013:KSAI03



UNIVERSITY OF BORÅS
SCHOOL OF BUSINESS AND IT

Systemarkitekturutbildningen är en kandidatutbildning med fokus på programutveckling. Utbildningen ger studenterna god bredd inom traditionell program- och systemutveckling, samt en spets mot modern utveckling för webben, mobila enheter och spel. Systemarkitekten blir en tekniskt skicklig och mycket bred programutvecklare. Typiska roller är därför programmerare och lösningsarkitekt. Styrkan hos utbildningen är främst bredden på de mjukvaruprojekt den färdige studenten är förberedd för. Efter examen skall systemarkitekter fungera dels som självständiga programutvecklare och dels som medarbetare i en större utvecklingsgrupp, vilket innebär förtrogenhet med olika arbetssätt inom programutveckling.

I utbildningen läggs stor vikt vid användning av de senaste teknikerna, miljöerna, verktygen och metoderna. Tillsammans med ovanstående teoretiska grund innebär detta att systemarkitekter skall vara anställningsbara som programutvecklare direkt efter examen. Det är lika naturligt för en nyutexaminerad systemarkitekt att arbeta som programutvecklare på ett stort företags IT-avdelning, som en konsultfirma. Systemarkitekten är också lämpad att arbeta inom teknik- och idédrivna verksamheter, vilka till exempel kan vara spelutveckling, webbapplikationer eller mobila tjänster.

Syftet med examensarbetet på systemarkitekturutbildningen är att studenten skall visa förmåga att delta i forsknings- eller utvecklingsarbete och därigenom bidra till kunskapsutvecklingen inom ämnet och avrapportera detta på ett vetenskapligt sätt. Således måste de projekt som utförs ha tillräcklig vetenskaplig och/eller innovativ höjd för att generera ny och generellt intressant kunskap.

Examensarbetet genomförs vanligen i samarbete med en extern uppdragsgivare eller forskningsgrupp. Det huvudsakliga resultatet utgörs av en skriftlig rapport på engelska eller svenska, samt eventuell produkt (t.ex. programvara eller rapport) levererad till extern uppdragsgivare. I examinationen ingår även presentation av arbetet, samt muntlig och skriftlig opposition på ett annat examensarbete vid ett examinationsseminarium. Examensarbetet bedöms och betygssätts baserat på delarna ovan, specifikt tas även hänsyn till kvaliteten på eventuell framtagen mjukvara. Examinator rådfrågar handledare och eventuell extern kontaktperson vid betygssättning.

HÖGSKOLAN I BORÅS
INSTITUTIONEN HANDELS- OCH IT-HÖGSKOLAN

BESÖKSADRESS: JÄRNVÄGSGATAN 5 · POSTADRESS: ALLÉGATAN 1, 501 90 BORÅS
TFN: 033-435 40 00 · E-POST: INST.HIT@HB.SE · WEBB: WWW.HB.SE/HIT

Svensk titel: Collaborative Filtering för att välja spelnivåer

Engelsk titel: Collaborative Filtering for choosing game levels

Utgivningsår: 2013

Författare: Fredrik Dahlberg, Mathias Söderqvist

Handledare: Rikard König

Abstract
More and more games are welcoming user generated content, often resulting in more material then any user can experience. What the unique user want, is for the material most suited for her, to be presented.

The study was conducted with the design science methodology, and using this methodology an artifact was created. With the help of the developed artifact, a platform game that is both simple to understand and play, could a data set be collected from various players. Data were built by the users after each completed level, explicitly allowed to leave his or her vote on a scale between 1 and 5.

With the introduction of collaborative filtering to recommend levels, by using the collected user ratings data of previous played levels and comparing those with the ratings of other players an expected rating on a specific level is estimated. By comparing different collaborative filtering algorithms, the experiments gave us an algorithm that performed with the best result for the platform game.

The result shows that by introducing collaborative filtering, a more precise estimates of a user's future ratings can be made than by just using the levels average rating. The result concludes that by introducing collaborative filtering, each user receives a more enjoyable game and provides the possibility to customize the gaming experience for a unique user and thus enhance their experience.

Keywords: Collaborative Filtering, game, item-based, user-based

Sammanfattning
Fler och fler spel öppnas upp för användargenererat innehåll, vilket ofta resulterar i större mängder material än vad en ensam spelare kan utnyttja. Den unika spelare vill ta del av det som passar just dennes smak.

Studien genomfördes med designforskning som metodval och med hjälp av denna metod skapades en artefakt. Med hjälp av den utvecklade artefakten, ett plattformspel som är både enkelt att förstå och spela, kunde en datamängd samlas in ifrån olika spelare. Data byggdes upp av att användarna efter varje slutförd nivå, explicit fick lämna sitt betyg på nivån i en skala mellan 1 och 5.

Genom att introducera collaborative filtering och där låta användarens tidigare betyg jämföras med övriga användare kan en predicering av kommande betyg ges. Vid jämförelser av olika collaborative filtering-algoritmer kunde den mest lämpliga upptäckas och senare även användas.

Resultaten visar att mer precisa uppskattningar av kommande betyg kan göras av collaborative filteringen än genom att använda nivåns medelbetyg och resultaten leder därför till slutsatsen att collaborative filtering kan ge skräddarsydda spelupplevelser för en unik användare och således förhöja dennes spelupplevelse.

Nyckelord: Collaborative Filtering, spel, item-based, user-based

Innehållsförteckning

1. Inledning - 1 -
1.1 Introduktion - 1 -
1.2 Problemformulering - 2 -
1.3 Frågeställning - 2 -
1.3.1 Avgränsning - 2 -
1.4 Syfte - 3 -
2 Introduktion - 3 -
2.1 Collaborative Filtering - 3 -
2.1.1 User-based - 4 -
2.1.2 Item-based - 4 -
2.1.3 Implicit vs explicit - 4 -
2.1.4 Svårigheter med rekommendationssystem - 5 -
2.2 Beräkning av likheter - 6 -
2.2.1 Korrelations-baserad uträkning - 6 -
2.2.2 Cosinus-baserad uträkning av likheter - 7 -
2.2.3 Anpassad cosinus-uträkning av likheter - 7 -
2.3 Predicering - 7 -
2.3.1 Viktat medelvärde för Item-based CF - 7 -
2.3.2 Viktat medelvärde för User-based CF - 8 -
2.3.3 Regression - 8 -
2.4 K-Nearest neighbors - 9 -
2.5 Winsorizing - 9 -
2.6 Evaluering av datamängden - 9 -
3 Relaterat Arbete - 10 -
3.1 Amazon.com - 10 -
3.2 Rekommendationer - 11 -
3.3 Flow - 11 -
4 Metodval - 12 -
4.1 Insamling av kunskapsbas - 12 -
4.2 Grunder i designforskning - 12 -
4.3 Insamlingsmetod - 13 -
4.4 Etiska aspekter - 14 -
4.5 Datamängd - 14 -
5 Metodtillämpning - 14 -
5.1 Definiera mål och krav på artefakten - 14 -
5.2 Utveckling av artefakten - 16 -
5.2.1 XNA Platformer Starterkit - 16 -
5.2.2 Utveckling av spel - 16 -
5.2.3 Nivårepresentation - 18 -
5.2.4 Nivåframtagning - 18 -
5.2.5 Lagring av data - 21 -
5.2.6 Collaborative Filtering - 21 -
5.3 Demonstration av artefakten - 22 -
5.4 Korsvalidering - 23 -
5.5 Experiment - 23 -
5.5.1 Item-based - 23 -
5.5.2 User-based - 25 -
6 Resultat - 26 -
6.1 Medelvärde - 26 -
6.2 Item-based - 27 -
6.2.1 MAE - 28 -
6.3 User-based - 29 -

6.3.1 MAE ..... - 29 -
6.4 Item-based VS user-based ..... - 30 -
7 Analys ..... - 31 -
8 Slutsatser ..... - 31 -
9 Diskussion ..... - 32 -
9.1 Nyhetens behag ..... - 32 -
9.2 Svårigheter och eventuella förbättringsområden ..... - 32 -
9.3 Uppdatering av likhetsmatriser ..... - 33 -
9.4 Giltighet ..... - 34 -
9.4.1 Upprepningsbarhet ..... - 34 -
Studiens upprepningsbarhet evalueras utifrån möjligheter för andra intressenter att vidareutveckla studien för att lösa problem av liknande karaktär. Med studiens detaljerade beskrivning samt medföljande källkod och testdata anses studien vara fullt replikerbar. Lösningen skulle kunna appliceras i andra fält med liknande betygsättnings system. ..... - 34 -
9.4.2 Representativitet ..... - 34 -
9.4.3 Reliabilitet ..... - 34 -
9.4.4 Validitet ..... - 34 -
10 Referenser ..... - 35 -
11 Appendix ..... - 37 -

1. Inledning

1.1 Introduktion

Det finns i dagsläget mängder med olika typer av data- och TV-spel. Liksom litteratur, film och musik kan även dessa spel delas in i olika genrer som tilltalar olika kategorier av människor. Exempelvis kan en spelgenre som first person shooter (FPS) tilltala spelare som tycker om actionfyllda sekvenser där spelarens reflexer sätts på prov, medan pusselgenren mer tilltalar spelare som uppskattar problemlösning där logik, strategi och mönsterigenkänning i större grad används.

I takt med att spel blivit mycket dyrare att utveckla har kravet på att tilltala större målgrupper vuxit. Stora utvecklingskostnader för spel kräver hög försäljning för att resultera i positiva siffror. Spel som Little Big Planet 2 (LBP 2) försöker att undvika att kategoriseras i en specifik genre genom att tillåta många genrer att existera inom ett och samma spel. Generella verktyg för att skapa spelnivåer tillhandahålls för spelarna så att de mest ambitiösa och intresserade spelarna själva kan skapa de typer av nivåer som de tilltalas av. Även verktyg för att skapa och ta del av musikaliska och cineastiska verk finns tillgängliga för att än mer bredda målgruppen.

Även spel som mestadels rör sig inom en specifik genre ger ofta användarna möjlighet till att bidra till spelet i form av user-generated content eller player-generated content (PGC) som det också kallas i spelsammanhang. Hjälpmedel för spelare att kunna skapa spelnivåer utan förkunskaper (map editors) är vanligt förekommande i spel som till exempel LBP 1 och 2 samt CounterStrike, men även stöd för att skapa helt nya objekt (The Sims) eller ändra utseendet på befintliga objekt, så kallade skins (Minecraft). PGC har blivit såpass vanligt förekommande att den viktiga spelportalen Steam (över 2000 spel och 50 miljoner användare (Steam u.å.), (Gamereactor 2012)) har skapat en specifik kategori för detta under namnet Steam Workshop.

Topplistor skapas ofta för att presentera de PGC som många användare uppskattar. Nackdelen med topplistor grundat på hela användarbasens genomsnittliga betyg är att objekt som anses väldigt bra enligt en mindre grupp försvinner i mängden då de kanske inte tilltalar den större massan. Mer önskvärt hade varit att topplistorna hade varit skräddarsydda för den unika användaren där hänsyn tagits till dennes preferenser och personliga smak. Inom flera områden används collaborative filtering (CF) för att göra just detta och lämna rekommendationer utifrån en specifik användares preferenser. Utan att ofta vara medveten om det så berörs den vanliga internetanvändare dagligen av CF genom otaliga sajter som till exempel youtube, amazon och imdb.

CF är ett rekommendationssystem för att förutse en specifik användares åsikt om en produkt. Utgångspunkten är att en användare är mer benägen att ta emot en rekommendation från andra användare vars åsikter delas, än från användare med vitt skilda åsikter. Produkter som en användare tidigare betygsatt används för att finna de användare som delar samma preferenser. Man låter dessa användare gemensamt rekommendera en produkt som den *aktiva användaren* (användaren som vill få en rekommendation presenterad för sig) ännu inte betygsatt men som de gemensamt gett ett positivt betyg. Det leder till att produkter som genomgående fått dåliga betyg inte kommer att bli rekommenderade medan produkter som tagit emot blandad kritik kommer att ta hänsyn till aktuell användare och dennes åsikter.

Genom att använda CF för urval av användargenererade spelnivåer så skulle man möjligtvis kunna kringgå svårigheterna med att rekommendera nivåer till en unik användare även för mycket stora mängder nivåer. Bra nivåer som tilltalar en typ av användare kommer att lyftas fram specifikt för dem, medan de sämre nivåerna inte kommer att få tillräckligt bra betyg, ens av de intresserade, för att rekommenderas. Då användare får spela de nivåer som bäst lämpar sig för just henne förväntas spelupplevelsen bli bättre och användaren får en roligare upplevelse.

1.2 Problemformulering

Användargenererade spelnivåer används i spelindustrin som ett sätt att bredda ett spels utbud och för att öka spelets livslängd. Det leder i många fall till en stor mängd nivåer med väldigt blandad kvalité. Dessa nivåer tilltalar också olika typer av spelare i varierande grad. Problemet med många befintliga lösningar är dock att rekommendationerna av nivåer inte tar hänsyn till den unika användaren utan lämnar rekommendationer enbart utifrån vad den stora massan användare tycker om aktuella nivåer. Det leder i sin tur att användaren kan få spela nivåer som inte uppskattas men också att användaren går miste om nivåer som skulle uppskattats.

1.3 Frågeställning

Är det möjligt att göra användarens spelupplevelse roligare genom att använda collaborative filtering för att rekommendera användargenererade spelnivåer.

1.3.1 Avgränsning

Denna studie är avgränsat till memory-based collaborative filtering men utforskar både user- och item-based collaborative filtering. All data som har samlats in är avgränsad till explicit betygsättning på spelnivåer med ett heltalsbetyg från 1 som klassificeras som en dålig nivå till 5 som klassificeras som en bra nivå. Inom PGC är studien avgränsad till spelnivåer.

1.4 Syfte

Syftet med detta arbete är att introducera CF som rekommendationssystem för användargenererade nivåer inom data- och TV-spel och undersöka om detta leder till en förhöjd spelupplevelse för den specifika användaren. En artefakt har tagits fram i form av ett plattformsspel där de nivåer som presenteras för användarna är valda med hjälp av CF och den specifika användarens tidigare preferenser. Om det predicerade betyget ligger nära det faktiska betyg som användaren lämnar skulle CF lämpa sig väl för större spel med många nivåer och en stor användarbas.

2 Introduktion

2.1 Collaborative Filtering

Collaborative filtering (CF) används i många olika sammanhang för att rekommendera produkter, företag eller media som kan intressera en unik användare. CF kan implementeras på många olika sätt men de flesta sätt kategoriseras inom antingen användarbaserad (user-based) eller objektbaserad (item-based) CF.

Målet med CF är att predicera hur intresserad den aktiva användaren är av ett specifikt objekt, baserad på användarens tidigare betygssättning och åsikter från andra likasinnade användare (Sarwar 2001). Det objekt som prediceraras att intressera den aktiva användaren mest kan sedan rekommenderas.

Zhang (2009) beskriver hur CF-algoritmer använder en matris (figur 1) för att representera förhållandet mellan $\mathrm{m} \times \mathrm{n}$ user-item, rad *m* representerar *m-te* användaren, kolumn *n* representerar *n-te* objekt, varje cell $R_{i,j}$ representerar ett betyg på objekt *j* av användaren *i*, som kan uttryckas inom en viss numerisk skala, exempelvis $0 \leq \mathrm{Ri,j} \leq 5$.

	$Item_1$	…	$Item_j$	…	$Item_n$
$User_1$	$R_{1,1}$	…	$R_{1,j}$	…	$R_{1,n}$
…	…	…	…	…	…
$User_i$	$R_{i,1}$	…	$R_{i,j}$	…	$R_{i,n}$
…	…	…	…	…	…
$User_m$	$R_{m,1}$	…	$R_{m,j}$	…	$R_{m,n}$

Figur 1

2.1.1 User-based

User-based CF utnyttjar statistiska metoder för att hitta de användarna vars preferenser bäst stämmer överens med den aktiva användaren. När en grupp av användare har hittats använder user-based CF olika algoritmer för att kombinera preferenserna från klustret av användarna för att förutse en rekommendation för den aktiva användaren (Mu, Chen & Li 2010).

De grundläggande antaganden och idéerna för user-based:

- Om användare har eller har haft en överensstämmande smak, kommer användarna ha en överensstämmande smak även i framtiden
- Användarens preferenser förblir stabila och konsekventa över tiden

Detta leder till att i de fall då endast en av de två likasinnade användarna uttryckt sin åsikt om ett objekt så förutsätts det att åsikten om just det objektet även gäller för den andra användaren.

2.1.2 Item-based

Item-based CF använder sig av likheter mellan objekt till skillnad mot user-based CF som använder likheten mellan användare. Det förutsätts att en användare som gett positiva omdömen om en viss typ av objekt även är intresserad av liknande objekt. Genom att ha tillgång till hur två objekt förhåller sig till varandra kan en rekommendation ges till en användare med hänsyn till dennes tidigare åsikter om liknande objekt.

Item-based CF använder en uppsättning av items som en användare har betygsatt och beräknar hur lika dessa items är jämfört med mål item *i* (det objekt som prediceringen gäller för), därefter väljs de *k* mest lika items $\{ i_1 , i_2 , \dots , i_k \}$. Likheterna mellan uppsättningen av item och mål item beräknas $\{ s_{i1} , s_{i2} , \dots , s_{ik} \}$. En predicering baseras sedan på de items som är mest lika varandra (Sarwar 2001).

2.1.3 Implicit vs explicit

För att CF skall fungera korrekt så förutsätts det att tillgång finns till den aktiva användarens preferenser. Tillgången till dessa preferenser fås antingen implicit eller explicit. Åsikter som tas emot explicit fås genom att användaren medvetet beskriver sin åsikt om ett objekt efter att ha använt det. Implicita åsikter fås istället genom att analysera användarens handlingar och att översätta det till en åsikt eller preferens. För att få reda på en användares smak inom musik kan man därför antingen be användaren betygsätta det som lyssnas på och hitta en genre med genomgående positiva betyg (explicit). Man kan också registrera vilka låtar som blir uppspelade, antalet gånger de spelas och granska sökorden som används för att dra slutsatser om användarens preferenser (implicit). (Breese, Heckerman & Kardie 1998)

2.1.4 Svårigheter med rekommendationssystem

Även om CF används flitigt så finns det problem som kan behövas ta hänsyn till. Nedan följer de vanligaste problemen direkt kopplade till CF.

2.1.4.1 Cold start

Bobadilla (2012) beskriver cold start som ett problem som inträffar då det inte är möjligt att lämna en rekommendation på grund av avsaknad av tidigare omdömen. Det särskils på tre olika cold start-problem: ny användarbas, nytt objekt och ny användare. Svårigheterna med att hitta likartade användare eller objekt uppstår i samtliga fallen och leder till uteblivna eller dåliga rekommendationer.

2.1.4.2 Neighbor transitivity

Många databaser innehåller information om många miljoner objekt och användare. Med stora datamängder finns det en risk för att användare som är väldigt lika varandra inte identifieras som sådana på grund av det faktum att de aldrig betygsatt samma objekt. Det samma gäller för objekt då det kan vara ovanligt med användare som gett sin åsikt om både objekten. (Su & Khoshgoftaar 2009), (Claypoo, Gokhale, Miranda, Murnikov, Netes, Sartin 1999).

2.1.4.3 Synonymi

Förekomsten av synonymer i beskrivande av objekt ger upphov till sämre rekommendationer för den aktiva användaren. Vid många tillfällen då CF används kommer det att vara möjligt för samma eller väldigt lika objekt att förekomma mer än en gång. I fall då algoritmen som påvisar likhet hos objekt tar hänsyn till objektets namn så skulle “children film” och “children movie” ses som två olika objekt även om de i själva verket är identiska.

2.1.4.4 Gray sheep

Användare med en oregelbunden smak har ofta svårt att hitta andra användare i små eller medelstora användarbaser med liknande preferenser. Med oregelbunden menas i detta sammanhang användare som uppskattar delar av till exempel en genre men inte hela genren. Med större användarbas skulle exakta eller snarlika användare kunna hittas men med färre användare så matchar ingen eller för få för att lämna riktigt bra rekommendationer. Genom att inte vara helt överens eller oense med några andra användare kommer rekommendationerna för den aktiva användaren inte vara helt skräddarsydda. (Claypoo et al 1999)

2.1.4.5 Black sheep

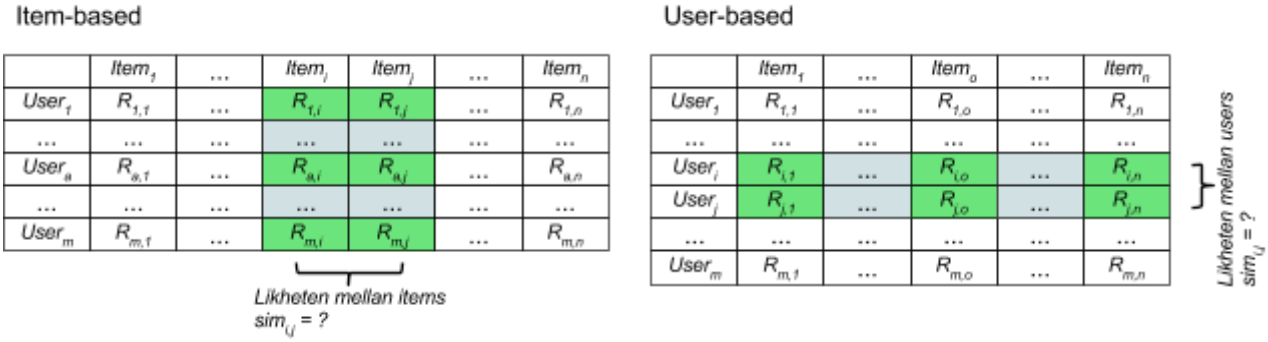
I motsatts till gray sheep så har black sheep-användare en så unik och egensinnig smak att det därför blir svårt att hitta andra användare med samma preferenser. Ett gray sheep är nästan ensam om att ha testat eller uppskattat ett objekt och kan därför inte ens jämföras med övriga användare. Dessa personer har svårigheter med att få rekommendationer även utanför automatiska system och ses därför som ett accepterat fel inom rekommendationssystem. (Su & Khoshgoftaar 2009)

2.1.4.6 Shilling attacks

Då användare själva har bidragit med eller har kopplingar till specifika objekt kommer de i viss utsträckning att försöka lyfta fram nämnt objekt. Det kan dels ske genom att överösa "sitt" objekt med positiva betyg eller att lämna negativa betyg på konkurrerande objekt. Lam och Riedl (2004) visade att user-based algoritmer i större utsträckning påverkades negativt av detta i jämförelse med item based-algoritmer.

2.2 Beräkning av likheter

Ett kritiskt steg i CF är att, beroende på vilken CF-algoritm som används, räkna ut likheten mellan antingen objekt eller användare, samt att välja ut de objekt/användare som är mest lika varandra. Figur 2 visar två matriser där kolumner representerar produkt och rader representerar användare. Cellen där en rad och kolumn möts återger betyget på aktuell nivå för aktuell användare. Skillnaden mellan user- och item-based är att i user-based sker beräkningen av likhet per kolumn och i item-based sker beräkning per rad i nämnda matriser (figur 2). Det finns många sätt att beräkna likheten mellan användare eller items, nedan presenteras de tre vanligaste.



Figur 2

2.2.1 Korrelations-baserad uträkning

Korrelation är ett värde för att beskriva hur två variabler förhåller sig till varandra och går därför att använda för både user- och item-baserad CF. Värdet ligger mellan -1 (maximalt negativt samband) och 1 (maximalt samband). Vid värdet 0 finns inget samband. Pearson-r (Rodgers & Nicewander 1988) är en vanlig algoritm som används för att ta fram detta värde. För att beräkna korrelationen (och då också likheten) mellan objekt *i* och *j*, med hjälp av Pearson-r, isoleras först de användare som har betygsatt de båda objekten, $U_{ij.}$ En enskild användare i mängden U_{ij} betecknas som *u*.

$$\mathrm{sim}(i,j) = \frac{\sum_{u \epsilon U_{ij}} \left(R_{u,i} - \bar{R}_i\right)\left(R_{u,j} - \bar{R}_j\right)}{\sqrt{\sum_{u \epsilon U_{ij}} \left(R_{u,i} - \bar{R}_i\right)^2} \sqrt{\sum_{u \epsilon U_{ij}} \left(R_{u,j} - \bar{R}_j\right)^2}}$$

Formel 1

I formel 1 ovan betecknas $R_{u,i}$ som betygsättningen av användare *u* på objektet *i*. $R_{u,j}$ betecknas som betygsättningen av användare *u* på objektet *j*. R_i är det genomsnittliga betyget på objekt *i*. R_j är det genomsnittliga betyget på objekt *j*.

2.2.2 Cosinus-baserad uträkning av likheter

I cosinus-baserad uträkning föreställs två objekt som vektorer i en *m*-dimensionell rymd. Likheten beräknas genom cosinus för vinkeln mellan dessa två vektorer. I $\mathrm{m} \times \mathrm{n}$ matrisen i formel 2 där varje cell representerar en användares betyg, betecknas likheten mellan objekten *i* och *j* som *sim(i, j)*.

$$\mathrm{sim(i,j)} = \cos(\vec{i}, \vec{j}) = \frac{\vec{i} \cdot \vec{j}}{\|\vec{i}\|^2 * \|\vec{j}\|^2}$$

Formel 2

2.2.3 Anpassad cosinus-uträkning av likheter

För att beräkna likheten mellan objekt med den anpassade cosinusformeln används skillnaden i betyg mellan olika användare. Likheten mellan objekt *i* och *j* visas i formel 3. Där $\bar{R}_u$ är medelbetygen för användare *u*.

$$\mathrm{sim(i,j)} = \frac{\sum_{u \epsilon U_{ij}} \left(R_{u,i} - \bar{R}_u\right)\left(R_{u,j} - \bar{R}_u\right)}{\sqrt{\sum_{u \epsilon U_{ij}} \left(R_{u,i} - \bar{R}_u\right)^2} \sqrt{\sum_{u \epsilon U_{ij}} \left(R_{u,j} - \bar{R}_u\right)^2}}$$

Formel 3

2.3 Predicering

Den viktigaste delen i en CF system är att predicera vilket objekt som visar sig att vara bäst lämpat för den aktiva användaren.

2.3.1 Viktat medelvärde för Item-based CF

Viktat medelvärde används för att fånga upp hur en aktiv användare betygsätter liknande objekt. Ett viktat medelvärde beräknas enligt (4) nedan, med summan av den så kallade vikten, vilket är antalet observationer av liknande objekt i sim matrisen multiplicerat med alla andra användares betygsättningar dividerat med summan av vikterna. I denna rapport används viktat medelvärde för att beräkna prediceringen för item/objektet *i* för användare *u* genom att beräkna summan av betygen som användaren angett för de items/objekt som är lika objektet *i*.
$P_{u,i}$ betecknar en användares *u* predicerade betyg för item/objektet *i*.
$S_{i,N}$ betecknar likheten mellan mål item/objektet *i* och dess motsvarande item/objekt *j*.

$$P_{u,i} = \frac{\sum_{all\ similar\ items,N}(S_{i,N} * R_{u,N})}{\sum_{all\ similar\ items,N}(|S_{i,N}|)}$$

Formel 4

2.3.2 Viktat medelvärde för User-based CF

Denna metod används för user-based CF och den fungerar genom att summera ihop den aktiva användarens medelbetyg, genom att använda hela uppsättningen av objekt som den aktiva användaren har betygsatt.

För att få fram en predicering för den aktiva användaren *a*, på ett specifikt objekt *i*, används (formel 5) för att få ett viktat medelvärde för alla betyg som den aktiva användaren *a* har betygsatt.

$$P_{a,i} = \bar{r_a} + \frac{\sum_{u \epsilon U}(r_{u,i} - \bar{r_a}) * w_{a,u}}{\sum_{u \epsilon U}|w_{a,u}|}$$

Formel 5

Där $\underline{r}_a$ och $\underline{r}_u$ är medelbetyget för användare *a* och användare *u* på hela uppsättningen med items/objekt. $w_{a,u}$ är vikten mellan användare *a* och användare *u*. Summeringen för alla användare $u \in U$ som har betygsatt objektet *i*.(Papagelisa & Plexousakisa 2005)

2.3.3 Regression

Detta tillvägagångsätt har likheter med det viktade medelvärdet men använder ett approximativt värde av betyget baserat på regressionsmodellen istället för att använda betyg av liknande objekt som det viktade medelvärdet. Sarwar (2001)

Idén är att använda samma formel som för det viktade medelvärdet, men istället för att använda liknande objekts "riktiga" betyg så använder metoden ett approximation betyg R'_N baserat på en linjär regressionsmodell. När målobjektet *i* och de liknande objekten *N*, betecknas som R_i och R_N, kan den linjära regressionsmodellen uttryckas enligt formel 6.

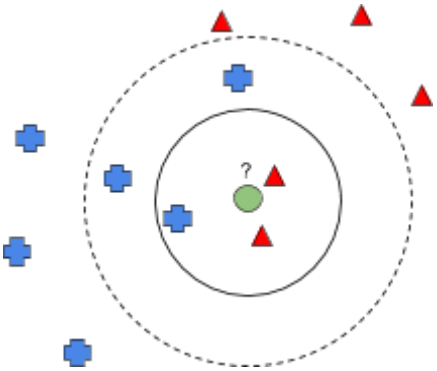
$$\bar{R'}_N = \alpha \bar{R}_\iota + \beta + \varepsilon$$

Formel 6

Parametrarna α och β bestäms genom att gå igenom de betygsvektorerna, ϵ är felet av regressionsmodellen.

2.4 K-Nearest neighbors

Vid klassificering av en okänd instans (figur 3) används ofta k-nearest neighbors för att placera nämnd instans i mest trolig klass. *k* är en variabel som informerar om antalet närmsta instanser att ta hänsyn till i klassificeringen. Vid *k* = 1 kommer instansen klassificeras till samma klass som närmsta granne. Ofta används ett högre *k* och då också en vikt (till exempel avstånd) som justerar hur påverkat slutresultatet blir av aktuell granne. I figur 3 visar den heldragna cirkeln vilka grannar som skulle tas hänsyn till vid k = 3 och den streckade cirkeln visar k = 5. Om inte en vikt används skulle k = 3 leda till att den gröna cirkeln klassificerades till röd triangel medan den vid k = 5 skulle ha klassificeras som ett blått plus.



Figur 3

2.5 Winsorizing

För att hantera värden som ligger utanför det förväntade eller långt ifrån majoriteten av värdena så används ofta winsorizing (ibland också skrivet winsorising). Det kan till exempel innebära att 90 procent av de neutrala värdena får agera intervall och att de övriga procenten får värdet av det tal, inom det neutrala intervallet, som ligger närmast. Det kan också användas för att tvinga värden att ligga inom ett förutbestämt intervall. Med ett intervall på 1 till 5 omvandlas då istället -2 till 1 och 9 till 5.

2.6 Evaluering av datamängden

Mean absolut error (MAE) är en teknik för att evaluera prediceringens träffsäkerhet. MAE fungerar enligt (formel 7) nedan.

$$\mathrm{MAE} = \frac{\sum_{U,S} \left| r_{u,s} - r_{\hat{u},s} \right|}{N}$$

Formel 7

$r_{u,s}$ är det predicerade värdet på objekt *s* av användare *u*. $r_{\hat{u},s}$ står för det riktiga värdet, *N* står för det totala antalet prediceringar och resultatet speglar i snitt hur långt ifrån prediceringen ligger det korrekta värdet över samtliga *N* prediceringar.

3 Relaterat Arbete

CF är relativt nytt och kommer ursprungligen ifrån arbete inom informationsfiltrering. Termen collaborative filtering myntades av Goldberg et al (1992), vilka var de första att använda CF-tekniken för att filtrera information. Systemet som skapades använde inte en automatisk CF-algoritm utan krävde att användarna skapade komplexa frågor (queries) i ett speciellt språk som skapades för systemet.

GroupLens (Resnick et al 1994) var först med att introducera en automatisk CF som använde en neighborhood-baserad algoritm, det vill säga en algoritm som givet en produkt och en användare kan förutse den mest sannolika åsikten för aktuell användare om produkten. GroupLens tillhandahöll personliga förutsägelser (prediceringar) för Usenets nyhetsartiklar. Usenets är ett sätt för grupper av användare att kommunicera med varandra över datanät, vie epost-liknande meddelanden.

Medler (2008) har gjort observationen att spel ofta görs enligt devisen "one-size-fits-all". Han menar att spelmakare förväntar sig att samtliga spelare vill uppleva det spelet de utvecklat på samma sätt. När tjänster inom annan media börjat göra personliga rekommendationer till en standard så kommer det snart förväntas i spel med. Medler (2008) studerar flera olika sätt att lämna rekommendationer och diskuterar också kring de många användningsområden inom spel där de skulle kunna användas, bland annat rekommendationer av spelnivåer baserat på tidigare åsikter. Implicit datainsamling förespråkas av den anledningen att frågan ifrån spelat om användarens betyg kan störa spelupplevelsen. Medler (2008) utför inga experiment utan lämnar det till framtida arbeten.

3.1 Amazon.com

Amazon.com rekommenderar produkter som deras användare kan tänkas uppskatta genom ett rekommendationssystem som använder en item-based CF. Den är enligt Linden, Smith, och York (2003) en skalbar metod som producerar rekommendationer av en hög kvalitet. Utöver detta erbjuder amazon även tjänster där användaren kan se vilka produkter som rekommenderas specifikt för aktuell användare samt möjligheter till att editera filtret för rekommendationer.

Linden, Smith, och York (2003) använder en item-based CF vars likhetsmatris inte behöver uppdateras i realtid varje gång data tillkommer utan kan beräknas i bakgrunden. Kostnaden för att beräkna likhetstabellen ligger i praktiken på O(NM), även om den i "worst-case" ligger på $O(N^2M)$ där N representerar antalen produkter och M antalet användare. Skalbarheten och prestandan hos item-based metoden kommer ifrån att den beräknas off-line och den kommer därför inte att påverkas av antalet användare. Antalet produkter som en användare betygsatt påverkar dock prestandan. Enligt Linden, Smith, och York (2003), är kvaliteten av rekommendationerna hög även med begränsat antal betygsättningar, något som utmärker den mot traditionella CF metoder, till exempel user-based CF. Linden, Smith, och York (2003) menar att lyckade rekommendationer till användare eventuell kommer att öka intäkterna för Amazon.com.

3.2 Rekommendationer

När spelutvecklare ger användare möjlighet att till exempel skapa egna nivåer så finns det en överhängande risk att ett väldigt stort antal nivåer kommer att skapas (LBP 2 hade i augusti 2012 sju miljoner nivåer (Media Molecule 2012)). Dessa nivåer kommer att vara i väldigt varierande kvalitet och en stor del av dem är kanske inte ens spelbara. Det finns ett behov att lyfta fram de nivåer som användare förväntas att tycka om men också ett ofrånkomligt behov av att filtrera bort de nivåer som inte tilltalar någon typ av spelare. Ett vanligt sätt att lyfta fram nivåer är att presentera de mest uppskattade nivåerna i en topplista som finns tillgänglig för alla användare. Detta används också inom all annan typ av media genom listor som amerikanska billboardlistan, imdb top 250 eller applikationstjänsten Google plays topplista. Gemensamt för dessa listor är att de representerar den stora massans åsikter och inte tar hänsyn till den unika personens preferenser. Utökningar till detta finns där sökord kan begränsa vad topplistan skall innehålla. På imdb.com finns möjligheter att välja den filmgenre som intresserar användaren och på så sätt få en lista något mer anpassad mot användarens smak. LBP låter användaren efter en spelad nivå koppla specifika ord till den. Detta är ord som vacker, rolig, kort, konstnärlig eller mörk. Utöver detta finns också möjligheten att placera ett hjärta på nivån som symboliserar användarens gillande. Topplistor skapas därefter baserat på dels enbart nivåns antal hjärtan men också för specifika ord som användaren själv väljer att söka på.

3.3 Flow

Enligt Sweetser och Wyeth (2005) beskriver Csikszentmihalyi *flow* som "en upplevelse som är så tillfredställande att människor är villiga att uppleva den för dess egen skull, med liten hänsyn till vad de får ut av det eller om det är svårt eller farligt." För majoriteten av alla spelare så inbringar inte spel några externa belöningar som pengar eller status. Sweetser och Wyeth menar istället att flow är en av de största anledningarna till att spel är så populärt.

Åtta element presenterades av Csikszentmihalyi för att uppnå flow

(1) en uppgift går att slutföra;

(2) förmågan att koncentrera sig på uppgiften;

(3) att koncentrationen är möjlig eftersom uppgiften har tydliga mål;

(4) att koncentrationen är möjlig eftersom uppgiften ger omedelbar feedback;

(5) förmåga att utöva en känsla av kontroll över handlingar;

(6) ett djupt men enkelt engagemang som tar bort medvetenheten om den frustration som upplevs i vardagslivet;

(7) oro för själv försvinner, men självkänsla stärkt efteråt, och

(8) betydelsen av varaktigheten för tiden ändras och genom detta kunna fånga användarens intresse och därigenom skapa en aktivitet som upplevs som rolig.

4 Metodval

I det här kapitlet diskuteras vilka forskningsmetoder som har används i studien.

Syftet med studien var att undersöka om det är möjligt att göra en användares spelupplevels roligare genom att använda CF för att rekommendera användargenererade spelnivåer. För att genomföra studien användes designforskning. “Design science is the study of artefacts in an intentional context. More precisely, design science is the study of artefacts and their environment as they are developed and used by people with the goal of solving practical problems. Studies in this field do not only answer knowledge questions but also develop solutions to practical problems.” Johannesson & Perjons (2012)
Designforskning användes för att på ett enkelt och strukturerat sätt kunna få in den kvantitativa data. Med hjälp av en interagerbar visuell produkt förväntas användarna lämna data som tydligare speglar deras åsikt.

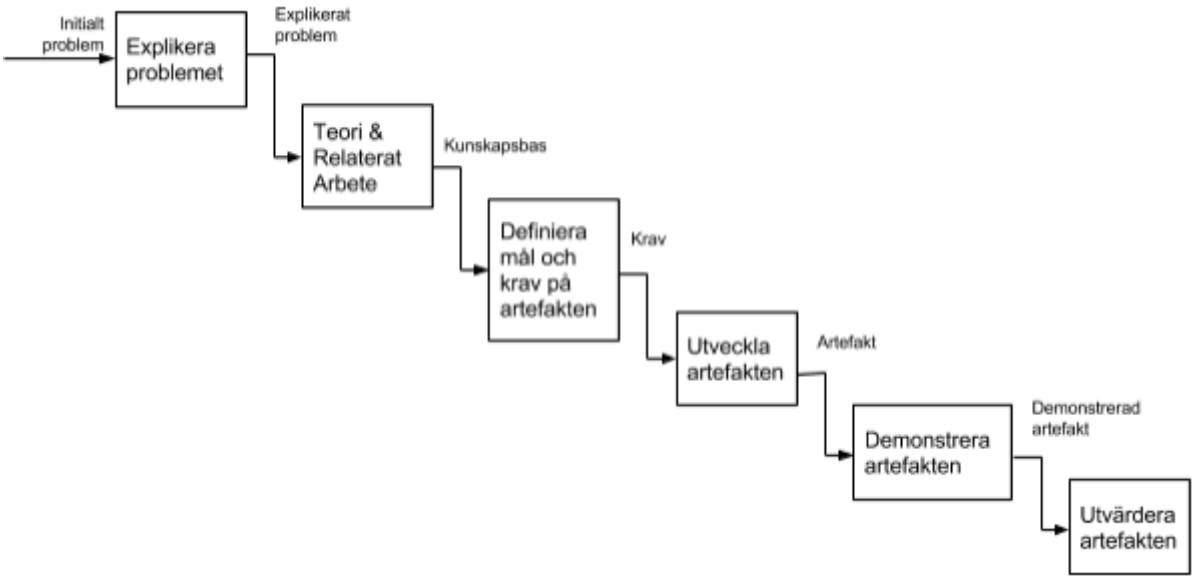
4.1 Insamling av kunskapsbas

För att kunna genomföra studien har en stor kunskapsbas krävts. Kunskapen samlades in genom en mängd litteraturreferenser. Referenserna samlades in ifrån artiklar ur vetenskapliga tidsskrifter, böcker och publicerade forskningsartiklar. Insamlingen underlättades med webbsidor som Web of science och Google scholar.
Kunskap inom spelutveckling har artikelförfattarna tillskansat sig från kurser inom spelutveckling samt dess kurslitteratur. Dessutom har ett genuint intresse för spel i allmänhet funnits hos artikelförfattarna sedan många år.

När insamlingen av litteraturen upplevdes komplett utvärderades de olika teknikerna som har samlats in. Teknikerna studerades grundligt för att utvecklingen skulle möjliggöras.

4.2 Grunder i designforskning

Vid genomförandet av rapporten används de riktlinjer som definieras enligt Johannesson och Perjons (2012) som ramverk för designforskningsprocessen. Metoden som Johannesson beskriver innefattar fem huvudsakliga aktiviteter som sträcker sig från problemformulering och kravhantering till utveckling av artefakt, demonstration och utvärdering. Utöver dessa fem aktiviteter har teori och relaterat arbete lagts till som nya aktiviteter för att läsaren skall få en lättare uppfattning vad som tidigare gjorts inom de relaterade områdena, dessa visas i figur 4.



Figur 4

4.3 Insamlingsmetod

Genom att låta användare spela artikelförfattarnas egna modifierade version av XNA starterkit har data samlats in. Användarna har blivit introducerade till spelet genom två stycken övningsnivåer som båda visar de grundläggande användningsområdena för de block som förekommer i spelet (som genomgås grundligt under punkt 5.2.4 Nivåframtagning). Samtidigt har det verbalt förklaras hur data som samlas in kommer att användas. CF har inte använts under datainsamling utan nivåerna har slumpats fram fritt efter de två övningsnivåerna. Alla nivåer måste ha slutförts innan någon nivå kan återupprepas.

För att slutföra en nivå har det krävs att användaren tar sig till mål eller misslyckats med detta fyra gånger. Möjligheten till att gå tillbaka och spela om en nivå har inte givits då det har prioriterats att användarna ska genomföra så många nivåer som möjligt under den begränsade tid vi har haft tillgång till dem. Efter slutförd nivå har användaren tvingats att betygsätta nivån utifrån hur kul användaren tyckte att nivån var och om användaren skulle vilja spela liknande nivåer i framtiden. Betygssättningen har skett i en skala från 1 till 5, där 1 är det lägsta betyget och 5 det högsta. Informationen om aktuell användare, nivå och betyg har sparats ner i en databas. För eventuella framtida arbeten har information om användarens spelsätt också sparats. Detta innefattar antalet dödsfall på nivån, samlade gems (samlarobjekt), antal hopp, antal dödade fiender av de olika typerna etc.

Då datainsamlingen hade påbörjats så förändrades inte spelet eller dess nivåer på något sätt. Även då mindre buggar och möjligheter att genomföra nivåer på sätt som artikelförfattarna inte avsett upptäcktes så gjordes inga ändringar. Detta för att garantera att de olika användarna som satt betyg på en nivå hade spelat exakt samma nivå med exakt samma förutsättningar.

4.4 Etiska aspekter

Vid skapandet av artefakten har det krävs en unik identifierare för varje användare. Användarens unika e-post användes för att spara data om hur användaren betygsatt nivåerna i spelet. Att be användare skriva in sin e-postadress kan klassas som känsligt och därför tilläts anonymitet genom påhittade e-postadresser som dock fortfarande var tvungna att kvalificera sig som unika identifierare. Användarna informerades om att e-postadressen inte kommer att användas för något annat syfte än att särskilja dem ifrån övriga användare. De upplystes också om hur deras betygsättningar skulle användas och även där i vilket syfte. (Forskningsetiska principer 2002)

4.5 Datamängd

För att verifiera effektiviteten av CF i artefakten som tagits fram har data som samlats in (4.3 Insamlingsmetod) använts. Datan består av användares betyg på nivåer i spelet. Den insamlade datamängden består utav 44 användare som totalt lämnat 1172 betyg mellan 1 - 5 på de 32 nivåerna som presenterats i en slumpmässig ordning för dem. Varje användare har betygsatt minst en nivå och nästan hälften har betygsatt samtliga 32 nivåer. (Dessa betyg för användarna finns i Appendix 3). En 10-fold-cross validering utfördes genom att dela upp datamängden i tio folds där en fold används som testmängd och resterande som träningsmängd. För item-based CF delas varje fold in med slumpmässigt utvalda användare och i user-based delas varje fold in med slumpmässigt utvalda nivåer. Metoden som används vid evalueringen av data beskrivs i Bellogín et al (2011) som *TrainingItems metod*. TrainingItems metoden fungera genom att ingen information om testmängden existerar. Därefter används träningsmängden för att evaluera testmängden.

5 Metodtillämpning

Här beskrivs hur vi förhåller oss till Design science riktlinjer.

5.1 Definiera mål och krav på artefakten

För att utforska frågeställningen som tagits upp i 1.3 Frågeställning utvecklas en artefakt. Artefakten som utvecklas i denna studie är av typen som kallas för instansiering. IT-artefakten instansiering betyder att ett fungerande system utvecklas. Fördelen med metodvalet är att det finns någonting konkret att visa upp. Hevner, March, Park, och S. Ram (2004) förklarar att designforskning ska initieras från ett verksamhetsbehov. Samtidigt skall forskningen, enligt designforskningsprocessen, bygga på ett vetenskapligt behov.

Inom PGC är det vetenskapliga behovet avsaknaden av likande undersökningar och verksamhetsbehovet har identifierats genom att ha observerat spel som LBP som har över 7 miljoner användargenererade spelnivåer.
Artefakten som utvecklades är ett 2D-platforms spel med stöd för PGC i form utav nivåer till spelet. Kraven diskuterades fram genom diskussion med handledare med design science och flow som utgångspunkt. Dessa krav listas nedan.

1. En användare ska snabbt förstå spelets kontroller, det vill säga vad alla knappar på handkontrollen gör
    - Om en användare snabbt kan förstå spelet hinner användaren att spela igenom fler nivåer vilket genererar mera data till studien
2. Tydlig feedback skall ges till användaren när någonting viktigt sker
    - Med tydlig feedback menas att spelaren skall förstå när någonting har hänt i spelet, exempelvis när spelaren dör och fiender dör. Detta minskar missförstånd hos användaren och ger en känsla av flow enligt Csikszentmihalyi (1990)
3. Det skall finnas en stor variation av element i spelet
    - För att kunna dra nytta av CFs fördelar krävs det en stor variation av element i spelet för att kunna skapa olika typer av nivåer. Med många olika element kan många olika typer av nivåer skapas, vilket i sin tur tilltalar olika spelare.
4. Spelet skall vara tillräckligt visuellt tilltalande för att inte förvirra användaren.

Krav på collaborative filtering

5. Betygsättning från användaren på en viss nivå skall sparas
    - För att genomföra studien behöver användaren betygsätta nivåer som användaren har spelat, data används sedan för CF
6. Data om spelaren behöver sparas för att sedan användas för att utföra experiment
7. Beräkningen av CF får inte störa en spelares spelupplevelse.
    - Beräkningen av matrisen skulle ta alldeles för stor kraft om detta skulle ske varje gång någonting hade uppdaterats i spelet, exempelvis att en spelare betygsätter en ny nivå
8. Rekommendationerna skall generera ett högre snittbetyg än en slumpmässigt vald nivå.

Målet med artefakten är att skapa ett fungerande spel som implementerar collaborative filtering för att förhöja spelupplevelsen och göra den roligare.

5.2 Utveckling av artefakten

Nedan beskrivs hur artefakten har tagits fram och vidareutvecklats med en agil systemutvecklingsmetod samt hur kraven som ställts på artefakten hanteras.

5.2.1 XNA Platformer Starterkit

För att få tillgång till grafiska element och en stabil grund för ett plattformsspel användes Microsoft XNA Starterkit: Platformer (framöver kallat XNA starterkit) (Microsoft u.å.). XNA starterkit är ett plattformsspel i 2D som finns gratis att ladda ner ifrån "msdn.com". XNA starterkit har i Microsofts version simpel funktionalitet för bland annat fysik, kollision och animering. Det har stöd för PC, Xbox och Windows Phone. Det har visuellt tilltalande animationer, tillräckligt bra ljudeffekter och motsvarar på många sätt det de mål och krav som satts upp.



Figur 5

Spelet är uppbyggt utav ett bakomliggande rutnät som hanterar objektens position på skärmen. Nivåerna till spelet skapas via en texteditor där varje tecken representerar ett block. För att än mer smidigt kunna skapa nivåer så vidareutvecklades FinalMirages XNA Platformer Level Editor (Jeff Brown 2011). Programmet visualiserar nivån på det sättet som den kommer att presenteras på i spelet och tillåter användaren att placera ut de block som nivån ska innehålla på ett mer överskådligt sätt. Vid lagring av data översätter sedan programmet nivån som skapats till den matris som beskrivs under rubriken nivårepresentation.

5.2.2 Utveckling av spel

Utvecklingen av spelet genomfördes med en agil systemutvecklingsmetod där arbetet utfördes helt iterativt. De iterativa arbetet underlättade vidareutvecklingen av XNA starterkit, då det i orginalutförandet saknade funktionalitet och behövde omstruktureras. Systemutvecklingsmetoden gjorde det enklare att utföra det krav som togs fram tidigare i rapporten (se 4.2 Definiera mål och krav på artefakten), genom att arbeta med korta delreleaser av spelet. För att säkerställa implementeringen av kraven, användes ett antal testare som parallellt med utvecklingen fick lämna kritik på huruvida de tyckte att vi hade tolkat kraven. Nedan listas hur dessa krav hanterades för att säkerställa lösningen.

Krav 1: En användare ska snabbt förstå spelets kontroller

- Vid pausskärmen presenteras en bild som beskriver alla kontroller i spelet.
- Kritiken som gavs från testarna hjälpte att ta fram en lösning som tillfredsställde spelarna.

Krav 2: Tydlig feedback skall ges till användaren när någonting viktigt sker

- Utökade det grafiska användargränssnittet med en mätare som symboliserar spelarens liv.
- Orginalutförandet av spelet innehöll redan ljudfiler som används när en spelare eller fiende dör samt diverse ljudfiler som ger tydlig feedback när någonting viktigt i spelet sker.
- Utöver ljud och animationer användes menyer för att ge feedback när spelaren har kommit till nivåns slutpunkt.

Krav 3: Det skall finnas en stor variation av element i spelet

- Fienderna som ingick i orginalutförandet av starterkit gick inte att döda och alla typer av fiender patrullerade enbart. Detta beteende utökades till en tillståndsbaserad artificiell intelligens (AI). Det finns tre tillstånd som en fiende kan använda sig utav och dessa tillstånd är patrullerande (fienden vandrar omkring och letar efter spelaren), jaga (fienden springer efter spelaren i ett försöka att döda, om spelaren hoppar, hoppar även fienden) och attackera (fienden skjuter ett eldklot mot spelaren som dör vid kollision). Fiendens aktuella tillstånd väljs utifrån positionen i förhållande till spelaren och för att få en variation på fienderna så har inte alla fiendetyper samtliga tillstånd tillgängliga. Modifikationer har också gjort för att fiender ska dö då spelaren landar på fiendens huvud.
- Nya block introducerades för att möjliggöra mer varierande och utmanade nivåer. De som introducerades är block som kan förflyttas horisontellt eller vertikalt, dessa block ger möjlighet för spelaren att klämmas ihjäl mellan ett statiskt block och rörligt block, respektive två rörliga block. Eldblock som dödar spelaren vid beröring, block som faller efter en viss tid, block som går att plocka upp, bära med sig och placera samt block som går att förstöra på tre slag.
- Testarna upplevde en stor variation av element i spelet, dessa åsikter säkerställde att kravet hade uppfyllts.

Krav 4: Spelet skall vara tillräckligt visuellt tilltalande för att inte skrämma bort användaren.

- XNA starterkit var i grundutförandet tillräckligt visuellt tilltalande och de nya block och menyer som implementerats har hållit sig till samma grafiska stil och känsla som tidigare.
- De visuella i spelet tilltalade testarna genom att inte skapa någon förvirring om vad de olika elementen i spelet har för funktionalitet. Med detta anses kravet vara uppfyllt.

Krav 5: Betygsättning från användaren på en viss nivå skall sparas

- Spelet utökades med en startmeny som kräver inloggning för att tillåta spel. Inloggningen kräver en godkänd e-post som sedan används för att en användare skall få en unik spelarprofil. Användarens betyg på de olika nivåerna knyts sedan automatiskt till den unika spelarprofilen. Utöver detta skapades en pausmeny, samt en meny som tar emot användarens betyg då en nivå spelats klart.
- Detta krav kontrollerades genom att utföra tester på implementationen. Testerna utfördes genom att skicka in ett betyg som input i spelet och därefter se att den förväntade betyget ligger lagrat i databasen.

Krav 6: Data om spelaren behöver sparas för att sedan användas för att utföra experiment

- Spelarens poäng på respektive avklarad nivå räknas numera ut med hänsyn till antalet dödade fiender, tillgänglig tid och antalet liv som är kvar vid målgång samt antalet gems som spelaren hunnit plockat upp. Poängen efter avslutad nivå sparas i en topplista som rankar de fem spelare med högst poäng.
- Detta krav kontrollerades genom att utföra tester på implementationen. Testerna utfördes genom att spela en nivå och manuellt skriva ned vilka saker som genomfördes under spelets gång. Därefter jämföra mot lagrade data i databasen.

Krav 7: Beräkningen av CF får inte störa en spelares spelupplevelse.

- Detta krav har ej blivit kontrollerat, då den mängd data som genererats vid insamlingen ej påverkat spelupplevelsen tillräckligt för att upplevas som ett problem.
- Om detta skulle bli ett framtida problem är rekommendationen att dessa beräkningar görs vid en offline-beräkning av likhetsmatriser

Krav 8: Rekommendationerna skall generara ett högre snittbetyg än en slumpmässigt vald nivå.

- Detta krav kontrollerades genom att utföra experiment för att säkerställa lösningen.

5.2.3 Nivårepresentation

Nivåerna representeras som en matris i en textsträng där mellanslag representerar radbyte och övriga tecken motsvarar en cell i matrisen och ett block i spelet. (Se appendix 1.)

5.2.4 Nivåframtagning

I originalutförandet av spelet skapades nivåer i en textfil som sedan inkluderas i projektet och laddades in genom att ange i koden vilken textfil som skulle inkluderas. Denna lösning ansågs inte optimal då det skulle ta alldeles för lång tid att skapa nivåer samt att det blir svårt att få en överblick huruvida nivån går att klara av eller ej. Detta problem åtgärdades genom att inkludera en nivå editor som justerades för att tillgodose de nya blocken som implementerades i modifikationen av XNA starterkit. Nivåerna sparas i en databas vilket gör det lätt att skapa nya, ändra och ta bort nivåer. Nivåerna som sparas i databasen laddas automatiskt in i spelet.

Vid framtagandet av nivåer till spelet har fokus legat på variation, både i sättet som nivån kommer att spelas på och i svårighetsgrader. Nivåer har skapats där de till antalet relativt få typer av block har utnyttjats på sätt som inte är självklara.

- När block som rör sig horisontellt placeras i en kolumn kommer de att fungera som en vägg som trycker spelaren åt ett specifikt håll och begränsar möjligheten till att stanna upp och fundera. Placeras dessa block istället i två kolumner med spelaren emellan sig så kommer spelaren begränsas än mer och spela nivån i den takt som nivåskaparen avsett. Det leder till nivåer som gynnar stresståliga spelare och som blir mindre förlåtande mot mindre stresståliga.

- Placeras vertikala block i rad med spelaren ovanpå fungerar de som ett golv som stadigt lyfter. Dessa nivåer kommer att begränsa framkomligheten på nivån då målet inte går att nås förrän golvet är i rätt position. Omvänt, då taket istället sjunker, tvingar spelaren att förflytta sig nedåt för att undvika att klämmas ihjäl.
- Gems används för att uppmuntra och utmana spelaren till att ta alternativa vägar som inte leder direkt mot mål. Då möjligheten ges till spelaren att antingen ta den enklare, mer säkra vägen till mål eller att samla gems antas valet spegla vilken kategori av spelare som spelaren tillhör.
- Nivåer som låter spelaren falla fritt ner mot mål samtidigt som eld tvingar spelaren att navigera sig runt den.
- Genom att ge tillgång till placerbara block låter man spelaren själv bygga sin väg mot målet. Mängden placerbara block som spelaren har tillgång till är direkt kopplad till svårigheten på nivån.
- Att använda fallande istället för flytande block kan leda till ökad svårighetsgrad på nivån samt förhindrande av *backtracking* (möjlighet att ta sig tillbaka till plats man tidigare besökt). Det leder till oförlåtliga nivåer där misstag kan leda till att det inte längre går att slutföra nivån utan att återstarta den.

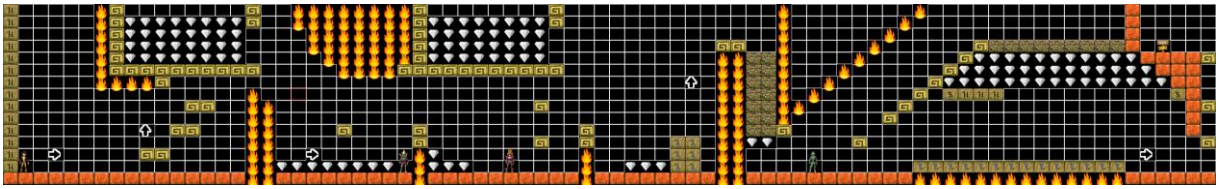
Många av nivåerna som har skapats kombinerar dessa nivåegenskaper samtidigt som de utnyttjar de mer naturliga användningsområdena för respektive block. Även renodlade klassiska plattformsnivåer har skapats. 34 nivåer har skapats men endast 32 har används för att samla in data, de kvarvarande två nivåerna har används som inledande handledning för användare att lära känna kontroll, olika block, fiender och fysik. De 32 nivåerna som användarna fått betygsätta har delats upp i olika kategorier utifrån deras mest tongivande inslag. Målet vid skapandet av nivåerna har varit att till varje kategori minst kunna knyta två stycken nivåer. Svårighetsgraden på nivåerna har inte påverkat vid indelningen.

Nedan följer de 13 olika kategorierna samt vilka nivåer som tillhör respektive kategori enligt artikelförfattarna:

1. Klassiska vertikala plattformsnivåer
    - 1.1. Classic Vertical 1
    - 1.2. Classic Vertical 2
    - 1.3. Classic Vertical 3
2. Klassiska horisontala plattformsnivåer
    - 2.1. Classic Horizontal 1
    - 2.2. Classic Horizontal 2
    - 2.3. Classic Horizontal 3
3. Klassiska plattformsnivåer med tongivande rörliga block
    - 3.1. Influental Moving Blocks 1
    - 3.2. Influental Moving Blocks 2
    - 3.3. Influental Moving Blocks 3
4. Nivåer med val mellan enkel eller utmanande väg
    - 4.1. Decisions 1
    - 4.2. Decisions 2

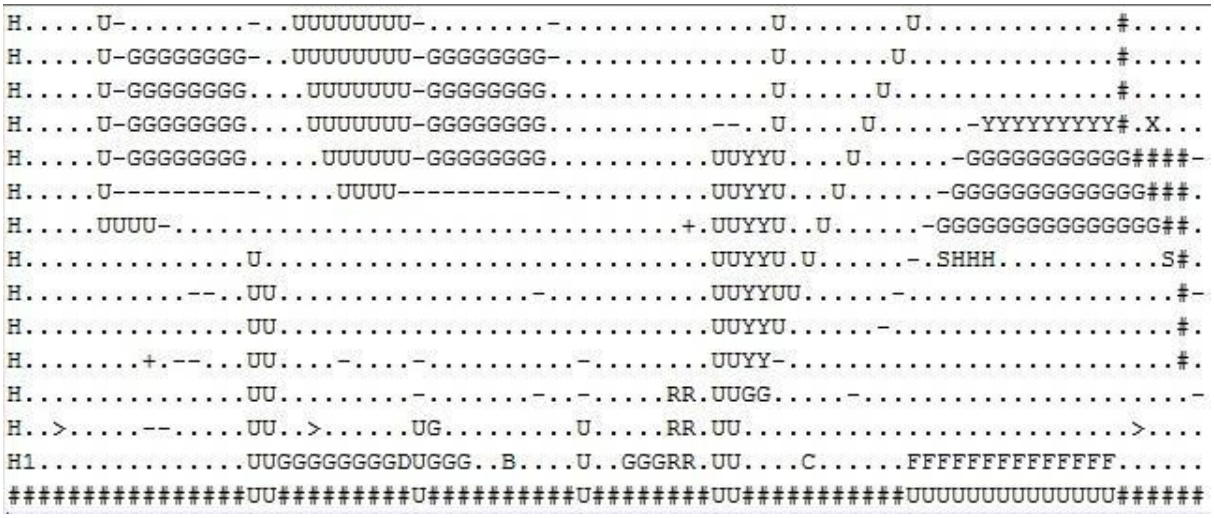
5. Nivåer med fall
    - 5.1. Falling 1
    - 5.2. Falling 2
    - 5.3. Falling 3
6. Nivåer med tongivande placerbara block
    - 6.1. Influential Placeable Blocks 1
    - 6.2. Influential Placeable Blocks 2
    - 6.3. Influential Placeable Blocks 3
7. Horisontella nivåer som kräver rörliga horisontella
    - 7.1. Forced Moving Blocks 1
    - 7.2. Forced Moving Blocks 2
    - 7.3. Forced Moving Blocks 3
8. Nivåer med vertikalt rörligt golv
    - 8.1. Moving Floor 1
    - 8.2. Moving Floor 2
9. Nivåer med horisontell rörlig vägg
    - 9.1. Moving Wall 1
    - 9.2. Moving Wall 2
10. Nivåer med två horisontella väggar
    - 10.1. Pressed Together 1
    - 10.2. Pressed Together 2
11. Nivåer med mycket gems
    - 11.1. Gems 1
    - 11.2. Gems 2
    - 11.3. Gems 3
12. Nivåer med övervägande fallande block
    - 12.1. Falling Blocks 1
    - 12.2. Falling Blocks 2
    - 12.3. Falling Blocks 3
13. Tutorials
    - 13.1. Tutorial 1
    - 13.2. Tutorial 2

Exempel på hur en färdig nivå kan se ut i level editorn visas i figur 6.



Figur 6

Och hur den sedan ser ut översatt till en textsträng där mellanslag representerar radbyte visas i figur 7.



Figur 7

Appendix 2 innehåller bilder på en nivå ur varje kategori för att tydligare illustrera hur de skiljer sig åt. Bilderna är hämtade ifrån nivågeneratorn men representeras på samma sätt i spelet.

5.2.5 Lagring av data

För att använda collaborative filtering krävs det att data om hur användaren har betygsatt en nivå finns tillgänglig. För detta används en MSSQL-databas. Mycket data sparades ner men i denna rapport användes endast information om betygsättning.

5.2.6 Collaborative Filtering

Under utvecklingen av artefakten har liten hänsyn tagits till de vanliga problem som uppstår inom CF. Cold start-problemen har inte påverkat under datainsamlingen av den anledningen att rekommendationer inte har lämnats utan slumpmässiga nivåer har presenterats för användarna. Problematiken med neighbor transitivity och till viss del synonymi gäller först vid mycket stora datasamlingar och är inte heller det någonting som påverkat framtagningen av artefakten. Över den relativt lilla datamängden som hanteras har full kontroll funnits över namngivning av nivåer och användare har enbart tillåtits en profil var. För att undersöka eventuell förekomst och hur rekommendationssystemet själv hanterar grey och black sheeps har ingen hänsyn tagits till dem under utvecklingen.

Då artikelförfattarna själva skapat samtliga nivåer i spelet har de inte fått lämna några röster för att motverka shilling attacks. Användare som har en personlig relation till nivåskaparen har blivit uppmuntrade att lämna uppriktiga betyg på nivåerna och vem som skapat en specifik nivå har inte framgått under datainsamlingen.

Collaborative filtering har implementerats med user- och item-based algoritmen, båda används senare i experimentet för att bestämma vilken av dessa CF algoritmer som presterar bäst. Ingen av algoritmerna kördes dock under datainsamlingen utan nivåerna presenterades i slumpvis ordning och data användes först i ett senare skede för att testa algoritmerna.
Ett av de krav som togs fram i kravhanteringen gällande collaborative filtering var att spara de betyg som användarna gav en viss nivå, detta löstes under (5.2.4 Lagring av data).

Kravet om att spara en användares betyg på en viss nivå krävs för att beräkna likheter mellan nivåer samt användare. Denna likhet beräknas enklast med pearson-r's korrelation, cosinus eller anpassad cosinus. I implementationen av CF används pearson-r's korrelation, då denna beskrivs av Herlocker, Konstan och Riedl (2002) som den mest "träffsäkra" tekniken för att beräkna likheten.

För att kunna rekommendera ett objekt till en användare måste en predicering om vilket betyg den aktiva användaren kommer att ge objektet göras. Om prediceringen visar sig att vara hög menas det att den utvalda användaren kommer att vara intresserad.
Sarwar (2001) nämner två prediceringsalgoritmer, dessa är regression och viktat medelvärde. Resultaten som Sarwar (2001) tar fram visar att regression fungerar bättre när datamängden saknar information, men att kvaliteten på prediceringen går nedåt när datamängden blir mer komplett. Eftersom att datamängden i studien är 83 % komplett togs beslutet att endast implementera viktat medelvärde för att hantera prediceringen av betygsättningen.

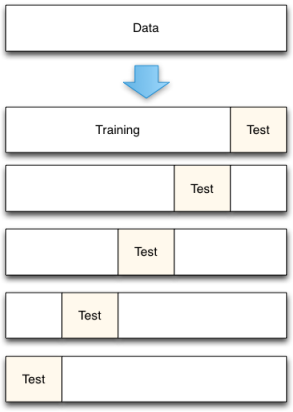
5.3 Demonstration av artefakten

Under utvecklingen av artefakten har systematisk testning genomförts. De tester som genomförts har enbart skett via observationer, de fel som uppstod dokumenterades och åtgärdades.

- Databastestning
    - Databasen har testats genom att observera att korrekt data har sparats i databasen, samt att korrekt data går att hämta ut genom korrekta databasanrop.
- Tester på XNA starterkit
    - När ny funktionalitet lagts till i spelet har det blivit testat genom att utsätta spelet för olika typer av scenarion som kan tänkas att orsaka problem. Det har också observerats att korrekt input kommer in till funktionen samt att programmet får korrekt förväntad output.
- Ett större test utfördes genom att åtta stycken testpersoner utföra ett vanligt scenario. Scenariot bestod utav att låta användarna logga in, spela igenom så många nivåer som de hade möjlighet till att spela samt att lämna betyg efter varje avklarad nivå. Efter testet fick testpersonerna berätta om deras synpunkter på artefakten och nödvändiga förändringar implementerades.

5.4 Korsvalidering

Korsvalidering eller cross validation är en statistisk metod för att utvärdera och jämföra inlärningsalgoritmer genom att dela upp datamängden i segment. Inlärningsalgoritmer är algoritmer som tränas upp med hjälp av bekräftad data för att senare dra slutsatser av obekräftad liknande data. I *k*-fold korsvalidering partitioneras data i *k* lika stora (eller nästa lika stora) segment eller folds. Därefter genomförs *k* iterationer där träning och test utförs så att inom varje iteration finns ett segment av data som döljs för validering medan resterande *k* - *1* segmenten används för träning. I figur 8 nedan visas ett exempel när *k* = 5.
(Refaeilzadeh, Tang & Liu 2008)



Figur 8

5.5 Experiment

Nedan följer detaljerade instruktioner på de experiment som utfördes för att utvärdera de två olika teknikerna inom CF. Utöver det beskrivs även experimentet för att få fram motsvarande information om nivåns medelvärde som prediceringsunderlag.

5.5.1 Item-based

Pseudokoden som följer nedan beskriver tillvägagångsättet för att undersöka hur lyckade prediceringar som CF-algoritmen utför och gäller för item-based CF.

```
(1) nLevelsPlayed = kNeighbors = Heltalslista med start på två och jämna
    intervaller till antalet nivåer - 1
Dela slumpmässigt in användarna i tio folds, folds


(2) För varje fold i folds låt fold vara testmängd och övriga träningsmängd
{
  (3) Ta fram likheten mellan alla nivåer utifrån träningsmängden, sim
  (4) För varje n i nLevelsPlayed
  {
    (5) För varje k i kNeighbors
    {
      (5) Utforska endast fall där k är mindre eller lika med n
```

```
      {
        För varje user i testmängden
        {
          (6) Utför kommande operationer fem gånger för att få fram ett
              medelvärde, r
          {
            För varje level som user spelat
            {
              (7) Välj slumpmässigt n nivåer som user spelat där level inte
                  kan bli vald, spelade
              (8) Välj de k nivåerna ur spelade som är mest lika level
                  enligt sim
              (9) Ta fram och spara prediktionen för level
              (10) Spara det verkliga betyget som user gett level
            }
            (11) Beräkna korrelationen för predicerade betyg jämfört med
                 verkliga betyg för user
            (12) Beräkna MAE för predicerade betyg jämfört med verkliga
                 betyg
          }
        }
      }
    }
  }
}
```

1. *nLevelsPlayed* och *kNeighbors* är två identiska listor som innehåller heltal i jämt intervall { 2, 4, 6, …, antal nivåer - 1 }
2. 10-fold-korsvalidering utförs
3. Pearson-r korrelationsalgoritm används och resultatet sparas i en matris med information om samtliga nivåer och hur de förhåller sig till varandra
4. Variabeln *n* styr hur många nivåer som det simuleras att den aktiva användaren har spelat då prediceringen görs
5. Variabeln *k* används för att justera hur många av de *n* nivåerna som skall användas i prediceringen. *k* kan inte vara större än *n* och de nivåer som väljs ut är de som har störst likhet mot den nivån som prediceringen gäller
6. Då det i punkt 7 innehåller ett slumpmoment så kan resultatet variera beroende på slumpen. För att få stabilare tester körs testen fem gånger och medelvärdet används
7. Slumpmässigt väljs *n* nivåer ut ur användarens spelade nivåer och övriga nivåer ses som ej spelade och utan betyg
8. kNearest neighbors väljs ifrån nivåerna som togs fram i steg 7. *k*:et i kNearest neighborsalgoritmen är det *k* som beskrevs i steg 5. Då nivån inte skall klassificeras blir resultat av kNearest neighbors enbart en lista på de nivåer som är mest lika aktuell nivå
9. Weighted sum för items används ifrån de *k* nivåerna som valdes ut i steg 8 och genom winsorizing garanteras att prediceringen ligger mellan 1 och 5
10. Det verkliga betyget som nivån fick av den aktiva användaren sparas för att senare jämföras mot det predicerade betyget

11. Pearson-r korrelationsalgoritm används återigen för att ta fram korrelationen mellan de predicerade betygen jämtemot facit
12. MAE används för att se hur långt ifrån facit prediceringarna är

5.5.2 User-based

Även om många av stegen går att återanvända för user-based CF så behöver experimentet för att testa user-based ändras om grundligt. Den största anledningen är att den likhetsmatrisen som byggs upp i mycket högre grad är kopplad till antalet nivåer som den aktiva användaren har spelat. Spelare kan få helt olika likheter enbart baserat på vilka nivåer som har spelats. *n* används även i denna pseudokod som en variabel för att beskriva antalet spelade nivåer för den aktiva användare medan *k* nu istället beskriver antalet liknande användare att basera prediceringen på.

```
nUserPlayedLevels = Heltalslista med start på två och jämna intervaller
  till antalet nivåer - 1
kNeigbors = Heltalslista med start på två och jämna intervaller
  till totala antalet användare - 1
För varje n i nUserPlayedLevels
{
  För varje user i users
  {
    (1) Utforska endast fall där user antal spelade nivåer är större än n
    {
      För varje k i kNeighbors
      {
        (2) Utför kommande operationer x gånger för att få fram ett
            medelvärde
        {
          För varje level i user spelade nivåer
          {
            (3) Välj n slumpmässiga nivåer, random, ur user spelade nivåer
            (4) Ta fram likheten, sim, mellan user och övriga users utifrån
                random
            (5) Välj de k mest liknande användarna från sim
            (6) Ta fram och spara prediktionen för level
            (7) Spara det verkliga betyget som user gett level
          }
          (8) Beräkna korrelationen för predicerade betyg jämfört med
              verkliga betyg för user
          (9) Beräkna MAE för predicerade betyg jämfört med verkliga betyg
        }
      }
    }
  }
}
```

1. Kan inte simulera *n* spelade nivåer om användarens spelade nivåer är mindre än *n*.
2. Används för att få fram ett medelvärde oavsett vilka nivåer som väljs i steg 3. Låt storleken på x vara beroende av *n* då det finns fler möjliga sätt att dela in en grupp nivåer i om *n* är ett litet tal. (Antalet spelade nivåer för user - *n*)

3. Slumpmässigt väljs *n* nivåer ut ur användarens spelade nivåer och övriga nivåer ses som ej spelade och utan betyg
4. Utifrån de slumpmässiga nivåerna som valdes ut i steg 3, ta fram likheten mellan user och övriga användare. Likheten tas fram genom Pearson-r korrelationsalgoritm
5. kNearest neighbors väljs utifrån de likheter som togs fram i steg 4
6. Viktat medelvärde för user-based ändvänds
7. Det verkliga betyget som nivån fick av den aktiva användaren sparas för att senare jämföras mot det predicerade betyget
8. Pearson-r korrelationsalgoritm används återigen för att ta fram korrelationen mellan de predicerade betygen jämtemot facit
9. MAE används för att se hur långt ifrån facit prediceringarna är

6 Resultat

Nedan presenteras de experimentresultat som tagits fram genom att implementera item- och user-based CF tillsammans med viktat medelvärde för beräkning av likheter samt pearsons korrelation för att beräkna prediceringen. Item-based kommer att jämföras mot user-based för att se vilken som presterar bäst utifrån den datamängd som funnits tillgänglig.

6.1 Medelvärde

För att enklare kunna relatera till de resultat som presenteras under item- och user-based CF har beräkningar också utförts för att predicera ett betyg enbart grundat på nivåns medelbetyg. Detta innebär att prediceringen inte tar hänsyn till den unika användare och dennes preferenser utan två användare som samtidigt ber om en predicering för samma nivå kommer alltid att få samma predicering.

MAE då prediceringen enbart är baserad på aktuell nivås medelbetyg är 1,1314.
I snitt så ligger alltså prediceringen ungefär 1,13 betygsenheter ifrån det betyg som användaren faktiskt lämnade. Om användarens faktiska betyg var tre så är den genomsnittliga prediceringen i detta fallet antingen på 1,87 eller 4,13. Då användare enbart kan lämna betyg mellan 1 och 5 så kan heller aldrig medelbetyget på en nivå ligga utanför den skalan och ingen winsorizing behövs alltså utföras.

6.2 Item-based

Likhetsmatrisen mellan nivåer som byggs upp av tillgänglig data är grunden i item-based CF. Under framtagningen av nivåerna gjordes det aktiva valet att dela in nivåer i olika kategorier och få nivåer inom samma kategori att likna varandra. Tabell 1 nedan beskriver korrelationen mellan användares betygsättning och då också hur lika nivåerna i respektive kategori är i snitt. Det mest relevanta och det som är färgkodat är hur nivåer inom samma kategori förhåller sig till varandra. Färgkodningen baseras på rött som det lägsta värdet, gult på det mellersta och grönt på det högsta. Endast värdena som visas i färg är med i framtagningen av färgkoden.

	Nivågenre	1	2	3	4	5	6	7	8	9	10	11	12
1	Vertikala plattformsnivåer	0,30	0,30	0,30	0,36	0,30	0,33	0,36	0,42	0,39	0,27	0,33	0,25
2	Horisontala plattformsnivåer	0,30	0,18	0,29	0,19	0,23	0,25	0,32	0,33	0,23	0,29	0,28	0,17
3	Tongivande rörliga block	0,30	0,29	0,32	0,21	0,33	0,24	0,32	0,46	0,23	0,22	0,23	0,16
4	Val mellan enkel eller utmanande väg	0,36	0,19	0,21	0,03	0,17	0,25	0,33	0,38	0,34	0,28	0,31	0,28
5	Fall	0,30	0,23	0,33	0,17	0,42	0,35	0,27	0,44	0,30	0,21	0,35	0,25
6	Tongivande placerbara block	0,33	0,25	0,24	0,25	0,35	0,59	0,32	0,43	0,26	0,31	0,36	0,28
7	Horisontella nivåer som kräver rörliga horisontell	0,36	0,32	0,32	0,33	0,17	0,32	0,33	0,36	0,39	0,34	0,31	0,31
8	Rörligt golv	0,42	0,33	0,46	0,38	0,44	0,43	0,36	0,58	0,37	0,44	0,36	0,29
9	Rörlig vägg	0,39	0,23	0,23	0,34	0,30	0,26	0,39	0,37	0,50	0,31	0,47	0,28
10	Två rörliga väggar	0,27	0,29	0,22	0,28	0,21	0,31	0,34	0,44	0,31	0,37	0,42	0,34
11	Mycket gems	0,33	0,28	0,23	0,31	0,35	0,36	0,31	0,36	0,47	0,42	0,30	0,35
12	Fallande block	0,25	0,17	0,16	0,28	0,25	0,28	0,31	0,29	0,28	0,34	0,35	0,24

Tabell 1

6.2.1 MAE

Tabell 2 visar hur långt ifrån det predicerade betyget det betyg som den aktiva användaren faktiskt lämnade ligger. Kolumnen längst till vänster beskriver *n* (antalet simulerade nivåer aktiva användaren spelat), översta raden *k* (antalet liknande nivåer som påverkar prediceringen) och cellen är som sagt MAE. Endast två decimaler presenteras för att öka läsligheten men färgkodningen visar de små skillnader som finns mellan de olika cellerna.

	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	31
2	1,13															
4	1,07	1,02														
6	1,02	0,98	1,00													
8	1,03	0,97	0,96	0,98												
10	1,03	0,97	0,95	0,95	0,97											
12	1,04	0,97	0,94	0,94	0,94	0,96										
14	1,03	0,97	0,94	0,93	0,94	0,93	0,95									
16	1,03	0,96	0,93	0,94	0,93	0,93	0,94	0,95								
18	1,04	0,96	0,94	0,93	0,93	0,93	0,93	0,93	0,95							
20	1,04	0,96	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,94						
22	1,05	0,96	0,93	0,93	0,93	0,92	0,92	0,93	0,93	0,93	0,95					
24	1,05	0,95	0,93	0,92	0,92	0,92	0,92	0,93	0,93	0,93	0,93	0,94				
26	1,06	0,96	0,93	0,92	0,92	0,92	0,92	0,92	0,93	0,93	0,93	0,93	0,94			
28	1,06	0,96	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,93	0,93	0,93	0,93	0,94		
30	1,06	0,96	0,92	0,92	0,92	0,92	0,92	0,92	0,93	0,93	0,93	0,93	0,93	0,93	0,94	
31	1,06	0,96	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,93	0,93	0,93	0,93	0,93	0,93	0,94

Tabell 2

Noterbart är att både *n* och *k* i item-based hanterar antal nivåer. Därför kan *k* aldrig vara större än *n* vilket förklarats mer utförligt i tidigare kapitel. I de fall då användaren inte har spelat *n* nivåer har *n* tagit värdet av antalet spelade nivåer - 1 (-1 på grund av att nivån som prediceringen gäller inte får vara en del av *n*). Värdena i tabell 2 är hur långt ifrån det korrekta betyget som prediceringen var och är det snitt som innefattar samtliga användare och nivåer. Som tidigare beskrivits körs varje predicering för användare och nivå flera gånger för att få ett mer korrekt medelvärde då slumpen spelar roll i vilka nivåer det simuleras att den aktiva användaren har spelat.

6.3 User-based

För user-based CF presenteras de resultat som togs fram genom ovan beskrivna experiment.

6.3.1 MAE

I tabell 3 nedan representeras *n* som rader och *k* som kolumner, värdena i varje cell är medelvärdet av MAE:en. I cell [2,2] syns ett MAE-värde på 1.1472, vilken innebär att med *n=2* och *k=2* ligger prediceringen 1,1472 i snitt, från den verkliga betygssättningen på nivån. Medan cell [28,22] tyder på att prediceringen ligger närmare den faktiska betygsättningen.

	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44
2	1,15	1,12	1,10	1,09	1,09	1,08	1,08	1,09	1,08	1,07	1,08	1,09	1,08	1,08	1,09	1,09	1,09	1,09	1,09	1,10	1,09	1,10
4	1,11	1,04	1,02	1,01	1,00	1,00	0,99	1,00	0,99	0,99	0,99	0,99	0,99	0,99	0,99	0,99	1,00	1,01	1,00	1,00	1,01	1,00
6	1,08	1,02	0,99	0,98	0,97	0,97	0,96	0,96	0,96	0,96	0,96	0,96	0,96	0,96	0,96	0,97	0,97	0,97	0,98	0,98	0,98	0,98
8	1,08	1,01	0,98	0,97	0,96	0,96	0,95	0,95	0,95	0,95	0,95	0,94	0,95	0,95	0,94	0,95	0,95	0,96	0,96	0,96	0,96	0,96
10	1,07	0,99	0,97	0,96	0,95	0,94	0,94	0,94	0,93	0,94	0,94	0,94	0,94	0,94	0,94	0,94	0,94	0,95	0,95	0,95	0,95	0,95
12	1,06	0,99	0,97	0,96	0,95	0,94	0,94	0,94	0,93	0,93	0,94	0,93	0,93	0,94	0,93	0,94	0,94	0,94	0,94	0,94	0,95	0,94
14	1,06	0,99	0,96	0,95	0,94	0,94	0,94	0,93	0,94	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,94	0,94	0,94	0,94	0,94
16	1,07	0,99	0,96	0,95	0,94	0,93	0,94	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,93	0,94	0,94	0,94	0,94
18	1,06	0,98	0,96	0,94	0,93	0,93	0,93	0,93	0,92	0,92	0,92	0,92	0,93	0,92	0,92	0,93	0,93	0,93	0,93	0,93	0,93	0,94
20	1,06	0,98	0,95	0,94	0,93	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,93	0,93	0,93	0,93	0,93
22	1,04	0,97	0,94	0,93	0,92	0,92	0,91	0,91	0,91	0,91	0,91	0,91	0,91	0,91	0,91	0,91	0,91	0,92	0,92	0,92	0,92	0,92
24	1,04	0,97	0,94	0,92	0,91	0,91	0,91	0,91	0,91	0,91	0,91	0,91	0,90	0,91	0,91	0,91	0,91	0,92	0,92	0,92	0,91	0,92
26	1,06	0,97	0,93	0,91	0,91	0,91	0,91	0,91	0,90	0,90	0,90	0,90	0,90	0,90	0,91	0,91	0,91	0,91	0,91	0,91	0,91	0,91
28	1,05	0,97	0,93	0,91	0,91	0,90	0,90	0,90	0,90	0,90	0,90	0,90	0,89	0,90	0,90	0,90	0,90	0,91	0,91	0,91	0,91	0,91
30	1,06	0,98	0,92	0,91	0,91	0,91	0,91	0,90	0,90	0,90	0,90	0,90	0,90	0,90	0,91	0,91	0,91	0,91	0,91	0,91	0,92	0,91
31	1,09	0,99	0,93	0,92	0,92	0,92	0,92	0,92	0,92	0,92	0,91	0,91	0,91	0,92	0,92	0,92	0,93	0,93	0,93	0,93	0,93	0,93

Tabell 3

6.4 Item-based VS user-based

Bilden nedan jämför MAE för item- och user-based för samtliga *n* och *k* som de har gemensamt. Siffran som är tjockare markerad beskriver *n* och därefter följer *k*. Färgkodningen under item och user visar vilken algoritm som fick det bästa resultatet för aktuella *n* och *k*. Diff står för differensen och grön färg visar tydlig vinst medan röd färg är att det var väldigt jämt. Item-based vann vid 60 tillfällen av 135 medan user-based vann vid 75 tillfällen.
I tabell 6 är *n* i tjock stil och *k* i normal stil.

n / k	Item	User	Diff
2			
2	1,130	1,147	0,017
4			
2	1,065	1,110	0,045
4	1,020	1,040	0,021
6			
2	1,024	1,085	0,060
4	0,984	1,015	0,032
6	1,005	0,995	0,010
8			
2	1,032	1,077	0,045
4	0,974	1,008	0,034
6	0,959	0,983	0,024
8	0,983	0,970	0,013
10			
2	1,030	1,074	0,044
4	0,975	0,995	0,020
6	0,947	0,972	0,024
8	0,948	0,960	0,012
10	0,969	0,951	0,018
12			
2	1,037	1,063	0,026
4	0,968	0,994	0,026
6	0,943	0,968	0,025
8	0,939	0,956	0,016
10	0,938	0,946	0,009
12	0,958	0,944	0,015
14			
2	1,032	1,065	0,033
4	0,969	0,990	0,021
6	0,943	0,958	0,016
8	0,933	0,953	0,021
10	0,941	0,940	0,001
12	0,932	0,940	0,008
14	0,952	0,936	0,017
16			
2	1,031	1,066	0,035
4	0,965	0,987	0,022
6	0,934	0,957	0,023
8	0,935	0,949	0,013
10	0,927	0,942	0,015
12	0,930	0,934	0,004
14	0,935	0,938	0,002
16	0,945	0,932	0,013

n / k	Item	User	Diff
18			
2	1,039	1,062	0,023
4	0,963	0,982	0,019
6	0,935	0,957	0,022
8	0,933	0,943	0,011
10	0,929	0,931	0,002
12	0,929	0,932	0,003
14	0,930	0,928	0,002
16	0,933	0,930	0,003
18	0,947	0,923	0,023
20			
2	1,040	1,059	0,019
4	0,960	0,979	0,018
6	0,933	0,950	0,017
8	0,931	0,941	0,010
10	0,928	0,929	0,001
12	0,929	0,923	0,006
14	0,929	0,918	0,011
16	0,928	0,917	0,011
18	0,933	0,916	0,017
20	0,943	0,917	0,026
22			
2	1,052	1,044	0,008
4	0,956	0,969	0,012
6	0,929	0,939	0,010
8	0,927	0,929	0,002
10	0,928	0,917	0,011
12	0,924	0,918	0,005
14	0,922	0,914	0,008
16	0,926	0,913	0,013
18	0,930	0,914	0,017
20	0,929	0,910	0,019
22	0,947	0,908	0,039

n / k	Item	User	Diff
24			
2	1,046	1,043	0,003
4	0,953	0,966	0,013
6	0,928	0,943	0,015
8	0,923	0,923	0,000
10	0,924	0,913	0,011
12	0,922	0,909	0,013
14	0,923	0,914	0,009
16	0,925	0,908	0,018
18	0,928	0,907	0,021
20	0,931	0,908	0,023
22	0,930	0,905	0,025
24	0,941	0,909	0,032
26			
2	1,060	1,062	0,002
4	0,960	0,969	0,009
6	0,929	0,928	0,001
8	0,922	0,910	0,012
10	0,920	0,912	0,008
12	0,920	0,912	0,008
14	0,923	0,910	0,013
16	0,923	0,907	0,016
18	0,928	0,903	0,026
20	0,929	0,904	0,026
22	0,929	0,905	0,024
24	0,931	0,901	0,030
26	0,941	0,904	0,037
28			
2	1,059	1,051	0,007
4	0,962	0,968	0,006
6	0,924	0,934	0,010
8	0,922	0,915	0,008
10	0,918	0,909	0,009
12	0,919	0,900	0,018
14	0,921	0,903	0,018
16	0,922	0,901	0,021
18	0,924	0,899	0,025
20	0,927	0,898	0,029
22	0,929	0,895	0,034
24	0,928	0,896	0,032
26	0,930	0,894	0,036
28	0,941	0,898	0,042

n / k	Item	User	Diff
30			
2	1,058	1,064	0,006
4	0,961	0,976	0,015
6	0,922	0,924	0,002
8	0,922	0,909	0,012
10	0,918	0,910	0,008
12	0,917	0,910	0,007
14	0,921	0,905	0,015
16	0,920	0,903	0,018
18	0,926	0,904	0,022
20	0,926	0,900	0,027
22	0,927	0,898	0,029
24	0,927	0,898	0,029
26	0,928	0,901	0,026
28	0,930	0,902	0,028
30	0,939	0,908	0,031
31			
2	1,062	1,095	0,033
4	0,963	0,989	0,026
6	0,920	0,930	0,010
8	0,922	0,918	0,004
10	0,916	0,922	0,005
12	0,917	0,924	0,007
14	0,920	0,921	0,000
16	0,920	0,922	0,001
18	0,925	0,918	0,007
20	0,925	0,917	0,009
22	0,928	0,915	0,013
24	0,929	0,912	0,017
26	0,928	0,915	0,013
28	0,929	0,917	0,013
30	0,933	0,922	0,011

Item	User
60	75

Tabell 4

7 Analys

Tabell 1 visar framförallt att den likheten som förväntades endast är tydlig i ett fåtal kategorier medan övriga kategorier ligger nära (eller under) kategorins medelvärde. Detta tros bero på nyhetens behag, det vill säga den möjligheten att en nivåtyp uppskattas som mest första gången den spelas. Detta utvecklas mer utförligt under slutsatser och diskussion.

Trots att likheterna mellan nivåer av samma typ inte speglas i användarnas betygsättning så lyckas CF hitta mönster i hur betygen är satta. Detta visar sig i både tabell 2 och 3 där det är tydligt att CF presterar bättre än nivåns medelbetyg. Det är en styrka hos CF att kunna uppfatta mönster som inte är uppenbara för människor och gör det också till ett mycket bra alternativ för väldigt stora datamängder.

Tabell 2 visar MAE för item-based CF och visar att det räcker med väldigt få spelade nivåer av en användare för att lämna prediceringar som är lika bra eller bättre än nivåns medelbetyg. Trots relativt små skillnader i MAE så visas det att prediceringen blir bättre med fler spelade nivåer och att *k*-värdet inte ska vara för högt.

Liknande iakttagelser kan göras i tabell 3 som visar MAE för user-based CF. Det blir stadigt bättre prediceringar då antalet spelade nivåer ökar även om den blir något sämre med 31 spelade nivåer gentemot 30. User-based CF är inte lika känslig för ändringar i *k*-värdet som item-based men de allra bästa prediceringarna uppnås när *k* är ungefär hälften av maximala k, precis som det gör för item-based.

Tabell 4 jämför de värdena som presenterades i tabell 2 och 3 och ger en tydlig bild av hur de olika teknikerna står sig emot varandra. Noterbart är att vid låga *n* så är item-based betydligt bättre än user-based. När *n* växer blir user-based bättre och bättre och går kring *n*=20 om item-based. Vi väldigt låga *k* fortsätter dock item-based att vara det bättre valet även om det stundtals är väldigt jämt.

8 Slutsatser

Studien visar framför allt att det är möjligt att lämna bättre rekommendationer genom att introducera CF än att använda den stora massans gemensamma rekommendation. Vid fler än fyra spelade nivåer för den aktiva användaren så lämnar både item- och user-based CF bättre rekommendationer än nivåns medelbetyg.

Det är relativt liten skillnad mellan hur bra rekommendationerna är mellan item- och user-based men som tidigare studier visat (och även vår uppmärksammat) så är det mycket mer tidseffektivt att använda item-based CF då likhetsmatrisen mycket mer sällan behövs räknas ut.

Egenskapen att bättre kunna förutse en unik användares åsikt om nivåer leder till att användaren kan få en skräddarsydd spelupplevelse som fokuserar på det användaren

uppskattar. Detta förhöjer den unika användarens spelupplevelse och svarar positivt på den ursprungliga frågeställningen.

9 Diskussion

9.1 Nyhetens behag

Observationer gjordes under datainsamlingen som tydde på att nivåer kunde få höga betyg om de introducerade element som användaren inte tidigare påträffat. Detta var betyg som inte var direkt kopplade till nivåtypen utan snarare till det faktum att det var någonting nytt för användaren. Efterföljande, likartade nivåer fick ett lägre betyg då nyhetens behag inte längre spelade in. Då användarna spelade nivåerna i slumpmässig ordning blir detta någonting som är svårt för prediceringen att hantera och en typ av neighbor transitivity. Två användare med identiska preferenser tycker egentligen att en nivå av varje typ är roligt men då de fått olika nivåer som introducerat nyheterna upptäcks inte deras röstningsmönster. Ju fler nivåer en användare spelar desto bättre borde rekommendation systemet hantera detta. För få nivåer har implementerats för att stödja detta antagande men med tio spelade nivåer inom samma typ så har det höga (möjligen oregelbundna) betyget vid första mötet med nivåtypen inte samma påverkan över den slutgiltiga rekommendationen.

Skulle det istället vara så att designen på nivåerna inte har så stora likheter som målet var stärks dock CF som teknik än mer. De likheter som tycktes självklara hos nivåskaparna stämde inte överens med mönstret som användarna röstade. Istället hittade CF nya mönster och gjorde det fortfarande bättre än nivåns medelbetyg.

9.2 Svårigheter och eventuella förbättringsområden

Trots uppmaningar till att använda hela betygskalan så fanns det ett antal användare som placerade en stor majoritet av sina betyg på en specifik siffra. Om detta berodde på bristande engagemang hos användaren eller det faktum att samtliga nivåer upplevdes på samma sätt framgår inte fullt. Samtliga nivåer för aktuell användare kommer att få prediceringar som ligger nära majoriteten av tidigare betyg. I fall då nivåerna upplevs på samma sätt och betygen är satta med eftertanke är detta en bra predicering även om rekommendationen inte blir så personlig som den skulle kunna bli. Användaren som istället ignorerat betygsättningen och valt första bästa för att ta sig till nästa nivå är svårare att hantera. Fortsätter detta betygsmönster kommer visserligen prediceringen ligga nära det satta betyget men användarens riktiga åsikt om nivån framkommer inte. Ett sätt att hantera detta vore att mer implicit samla in användarens åsikt eller att försöka hitta likheter till andra användare baserat på egenskaper hos spelaren istället för betygen som sätts. Användare som hoppar mycket, samlar alla gems men aldrig dödar fiender skulle kunna dela åsikter även inom vilka nivåer som uppskattas.

Nämnvärt i sammanhanget är hur viktigt det är att med olika medel få användaren mer medveten av fördelarna av att lämna uppriktiga och seriösa betyg. Genom att hjälpa sig själv få bra rekommendationer hjälper användarna även varandra och rekommendation systemet.

Även om resultaten från både item- och user based CF tydligt visar att tekniken fungerar inom plattformsspel anses prediceringarna kunnat bli ännu bättre om användarna haft mer tid på sig att bekanta sig med spelet innan de började lämna betyg. Dels hade problemet med nyhetens behag som tidigare nämndes inte blivit lika avgörande men framför allt hade nivåbetygen inte blivit så pass kopplade till det faktiska spelet som nu var fallet. Trots att två tutorial nivåer användes för att bekanta användaren med spelet har flera användare beskrivit att det varit svårt att lämna betyg de första nivåerna då det funnits för lite att jämföra med. I LBP uppmanas användaren att starta med de nivåer som spelskaparna själva utvecklat och där det inte finns någon möjlighet att rösta. Det eliminerar problem kopplade till brist på jämförelser och hanterar också många fall av nyhetens behag.

9.3 Uppdatering av likhetsmatriser

Då både item- och user-based CF använder sig av information om likhet mellan objekt/användare måste den informationen hålla sig uppdaterad. Här skiljer sig dock user- och item-based CF ifrån varandra. Varje betyg som sätts på en nivå förändrar hur likheten till andra nivåer är men inte tillräckligt mycket för att resultera i drastiska skillnader. För en användare, och då i första hand en relativt ny användare, kommer dock ett enskilt betyg ordentligt kunna förändra hur den användaren förhåller sig till övriga användare i systemet. Då testerna utfördes under detta arbete fanns tillräckligt få objekt och användare för att kunna ta fram en ny likhetsmatris vid varje ny rekommendation men detta är inte realistiskt i ett större sammanhang och detta är ett problem för framför allt user-based CF. Likheterna mellan nivåer är mycket mer statisk och lösningen på detta inom item-based CF skulle kunna vara att varje natt uppdatera likhetsmatrisen och under nästkommande dag använda den senast beräknade matrisen utan att uppdatera den. Men som tidigare nämnts så krävs det att en ny användare, gärna efter varje spelad nivå, får färska siffror som beskriver likheten till övriga användare.

I den slutgiltiga versionen av artikelförfattarnas spel har item-based CF implementerats med ett *k* på 12. Item-based CF på grund av att det enklare och mer tidseffektivt ska gå att skala upp till betydligt fler användare och nivåer och 12 för att det med tillgänglig data visade sig vara den andelen användare/nivåer som mest träffsäkert lyckades predicera den aktiva användarens betyg. Valet av *k* har i sammanhanget gjorts på väldigt lite data och det kan inte garanteras att det är det bästa valet när antalet nivåer och användare kraftigt har ökats. Det har i item-based CF dock tagits hänsyn till om användaren inte har spelat tillräckligt många nivåer för att använda det *k* som satts ut. Istället används då det maximala *k* som är möjligt grundat på användarens spelade nivåer. Rekommendationen som ges är att vid jämna intervall återutvärdera värdet på *k* genom att genomföra liknande tester som gjorts.

Det skulle mycket väl när antalet nivåer växer kunna visa sig att det bästa värdet på *k* är väldigt varierande beroende på antalet nivåer som den aktiva användaren har genomfört. Att dynamiskt anpassa *k* till den aktiva användarens spelade nivåer skulle därför kunna resultera i bättre resultat.

9.4 Giltighet

9.4.1 Upprepningsbarhet

Studiens upprepningsbarhet evalueras utifrån möjligheter för andra intressenter att vidareutveckla studien för att lösa problem av liknande karaktär. Med studiens detaljerade beskrivning samt medföljande källkod och testdata anses studien vara fullt replikerbar. Lösningen skulle kunna appliceras i andra fält med liknande betygsättnings system.

9.4.2 Representativitet

Studiens representativitet anger huruvida slutsatserna är generaliserbara.
Studiens slutsats visar på att det fullt generaliserbart att applicera artefakten för att rekommendera spelnivåer. Utöver att använda CF för att rekommendera spelnivåer talar slutsatserna för att artefakten är generaliserbar inom andra områden där användares åsikter lagras och kan användas för rekommendationer.

9.4.3 Reliabilitet

Tillförlitlighet i datainsamlingen och analysen med avseende på slumpmässiga variationer.
Studiens reliabilitet stärks genom att insamlingen av data skedde med en detaljerad genomgång av hur den insamlade data hanterats nämndes i 4.3 Insamlingsmetod. Detta säkerställer en representativ datamängd valts vilket medför att studien kan återskapas med insamlad data.

9.4.4 Validitet

Studiens validitet beskriver sammanhanget hur pass bra och korrekta de valda kriterier och kvalitetsdimensioner som studien tagit fram är för utvärdering av prototypen.
Resultatet som presenterades i studien är i stor del förslag på ett sätt att lösa problemet som presenterats Studiens stärks av att den prototyp som utvecklats i samband med studien är testad med verklig data.

10 Referenser

Bellogín, A., Castells, P. & Cantador, I. (2011). *Precision-Oriented Evaluation of Recommender Systems: An Algorithmic Comparison*, Proceedings of the fifth ACM conference on recommender systems, 10/2011, pp. 333-336

Bobadilla J., Ortega F., Hernando A. & Bernal J. (2012). A collaborative filtering approach to mitigate the new user cold start problem. *Knowledge-Based Systems*, 26, ss. 225-238.

Breese, J.S., Heckerman, D., Kadie, C. (1998). Empirical Analysis of Predictive Algorithms for Collaborative Filtering. *Proceedings of the Fourteenth conference on Uncertainty in artificial intelligence (UAI'98)*, ss. 43-52.

Claypool, M., Gokhale, A., Miranda, T., Murnikov, P., Netes, D. & Sartin, M. (1999). Combining Content-Based and Collaborative Filters in an Online Newspaper. *Proceedings of the ACM SIGIR '99 Workshop on Recommender Systems*

Forskningsetiska principer inom humanistisk-samhällsvetenskaplig forskning. (2002). Stockholm: Vetenskapsrådet, Tillgänglig på Internet: http://www.cm.se/webbshop_vr/pdfer/etikreglerhs.pdf

Gamereactor (2012-11-22)
http://www.gamereactor.se/nyheter/79691/50+miljoner+Steam-anv%E4ndare/ [2013-08-20]

Goldberg, D., Nichols, D., Oki, B.M. & Terry, D. ,(1992), Using collaborative filtering to weave an information tapestry. *Communications of the ACM*, 35(12), ss. 61-70.

Herlocker, J., Konstan, J. & Riedl, J., (2002). An empirical analysis of design choices in neighborhood-based collaborative filtering algorithms. *Information Retrieval*, 5(4), ss. 287-310.

Hevner, A., S. March, J. Park, and S. Ram (2004) Design science in information systems research, MIS Quarterly 28 (1), pp. 75–105

Jeff Brown. (2011). *XNA Platformer Level Editor* (Version 1.0) [Programvara]. Tillgänglig: https://bitbucket.org/FinalMirage/xna-platformer-level-editor [2013-08-22]

Jinbo, Z., Zhiqing, L., Bo, X. & Chuang, Z.,(2009), "An optimized item-based collaborative filtering recommendation algorithm". *IC-NIDC 2009. IEEE International Conference on Network Infrastructure and Digital Content, 2009*, vol., no., pp.414,418, 6-8 Nov. 2009

Johannesson, P. & Perjons, E. (2012). *A Design Science Primer*. CreateSpace Independent Publishing Platform.

Lam, S & Riedl, J. (2004). Shilling recommender systems for fun and profit. *Proceedings of the 13th international conference on World Wide Web* (WWW '04). ss. 393-402.

G. Linden, B. Smith, & J. York, (2003) Amazon.com Recommendations: Item-to-Item Collaborative Filtering. IEEE Internet Computing, ISSN 1089-7801, 2003, Volym 7, Nummer 1, pp. 76 – 80

Media Molecule (2012-08-06)
http://www.mediamolecule.com/blog/category/littlebigplanet/ [2013-08-22]

Medler, B. (2008). Using Recommendation Systems to Adapt Gameplay

Microsoft. (u.å.). *Starterkit: Platformer* (XNA Game Studio Version 3.1) [Programvara]. Tillgänglig: http://msdn.microsoft.com/en-us/library/dd254918(v=xnagamestudio.31).aspx [2013-08-22]

Mu, X., Chen, Y. & Li, T. (2010) *User-based collaborative filtering based on improved similarity algorithm*, Computer Science and Information Technology (ICCSIT), 2010 3rd IEEE International Conference on , vol.8, no., pp.76,80, 9-11 July 2010

Papagelisa, M. & Plexousakisa, D. (2005). Qualitative analysis of user-based and item-based prediction algorithms for recommendation agents. *Engineering Applications of Artificial Intelligence*, 18(7), ss. 781-789.

Refaeilzadeh, P., Tang, L. & Liu, H. (2008). *Cross-Validation*
http://www.cse.iitb.ac.in/~tarung/smt/papers_ppt/ency-cross-validation.pdf

Resnick, P. Iacovou, N. Suchak, M. Bergstrom, P. & Riedl, J. (1994). *GroupLens: an open architecture for collaborative filtering of netnews*. In Proceedings of the 1994 ACM conference on Computer supported cooperative work (CSCW '94). ACM, New York, NY, USA, ss. 175-186.

Rodgers, J.L. & Nicewander, W.A. (1988). Thirteen Ways to Look at the Correlation Coefficient. *The American Statistician*, 42(1), ss. 59-66.

Steam (u.å.)
http://store.steampowered.com/about/ [2013-08-20]

Su, X. & Khoshgoftaar, T.M. (2009). A Survey of Collaborative Filtering Techniques, *Advances in Artificial Intelligence*, 2009(no.4).

Sweetser, P. & P. Wyeth (2005). GameFlow: a model for evaluating player enjoyment in games. *Comput. Entertain.* 3(3), 3-3.

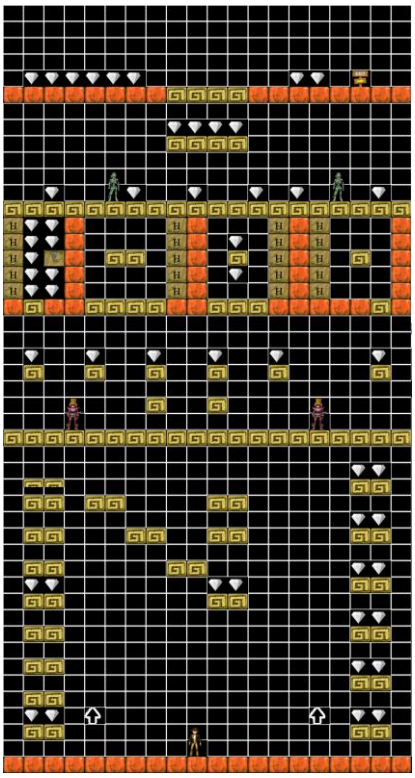
11 Appendix

Appendix 1 - Blocktyper

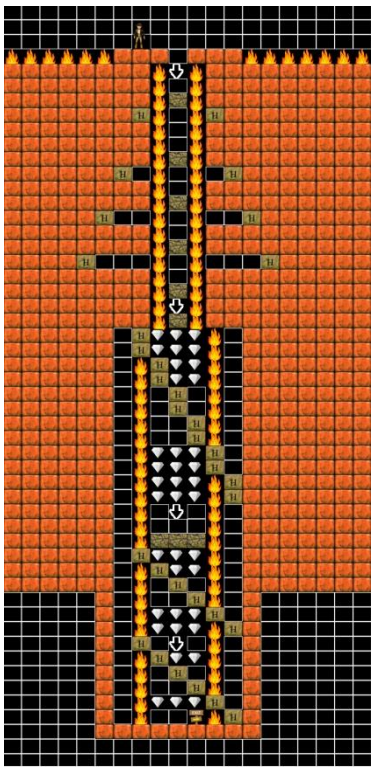
1	Spelaren	Punkten där spelaren startar på nivån
A, B, C, D	Fiender	Fyra olika typer av fiender där A kan jaga, B och C kan skjuta och D kan jaga och skjuta.
X	Mål	Punkten där spelaren slutför nivån och poängen räknas samman
-	Flytande block	Block som går att stå på men som i övrigt saknar kollision
#	Plattformsblock	Block som har kollision i alla riktningar
G	Gems	Samlingsobjekt som bidrar till slutpoängen på nivån
H, V	Rörliga block	Plattformsblock som rör sig. H förflyttar sig horisontellt medan V förflyttar sig vertikalt. Kommer att döda spelaren om denne kläms
S	Stoppblock	Osynligt block utan kollision som ändrar riktning (ej dimension) på rörliga block
<, >, +, /	Pilar	Block som representeras som pilar för att vägleda spelaren. Har ingen kollision.
R	Placerbara block	Flytande block som går att plocka upp, bära med sig och placera ut
F	Fallande block	Flytande block som faller strax efter att spelaren stått på det
Y	Förstörbara block	Plattformsblock som går att slå sönder om spelaren står på det
U	Eld	Dödar spelaren vid kollision
.	Avsaknad av block	Ingen funktionalitet

Appendix 2 – Speltyper och nivåer

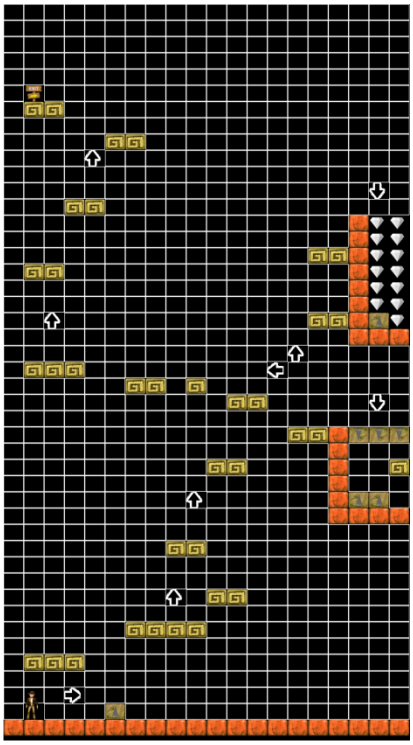
1. Klassiska vertikala plattformsnivåer



Figur 4 Classical Vertical 1

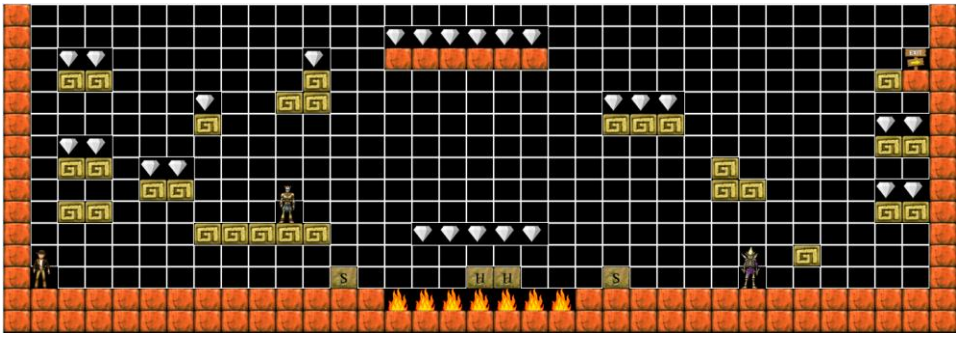


Figur 2 Classical Vertical 2

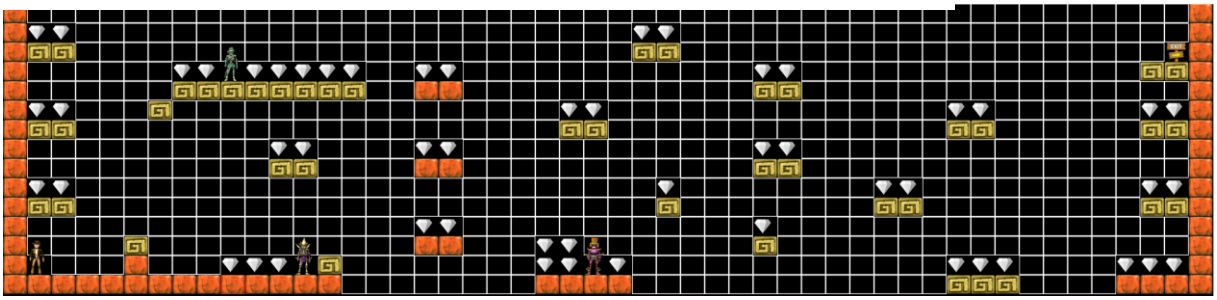


Figur 3 Classical Vertical 3

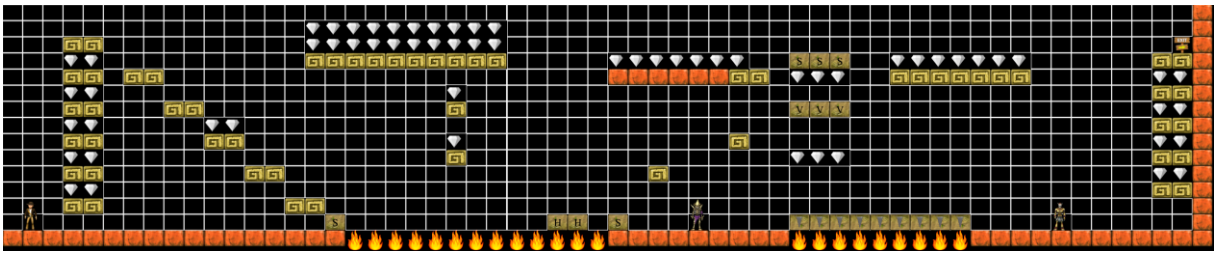
2. Klassiska horisontala plattformsnivåer



Figur 4 Classic Horizontal 1

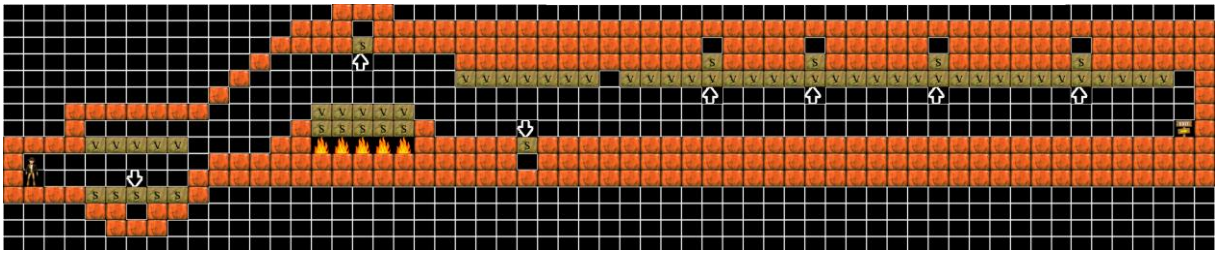


Figur 5 Classic Horizontal 2

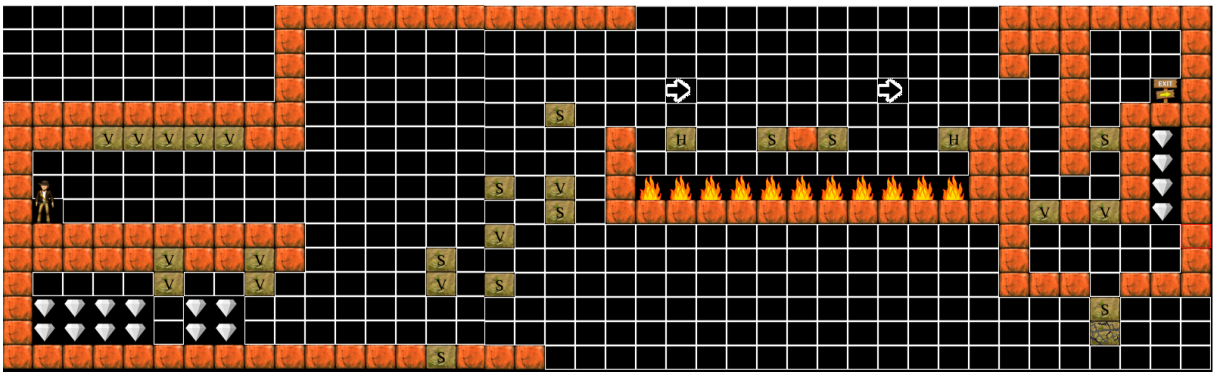


Figur 6 Classic Horizontal 3

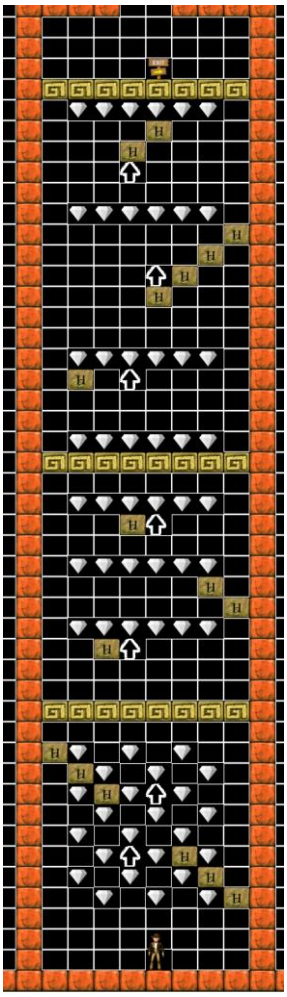
3. Klassiska plattformsnivåer med tongivande rörliga block



Figur 7 Influental Moving Blocks 1

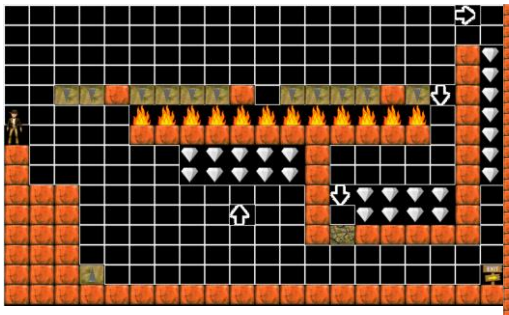


Figur 8 Influental Moving Blocks 2

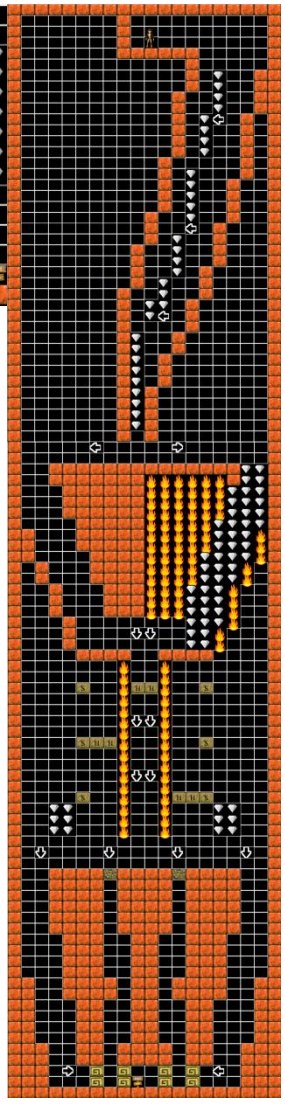


Figur 9 Influental Moving Blocks 3

4. Nivåer med val mellan enkel eller utmanande väg

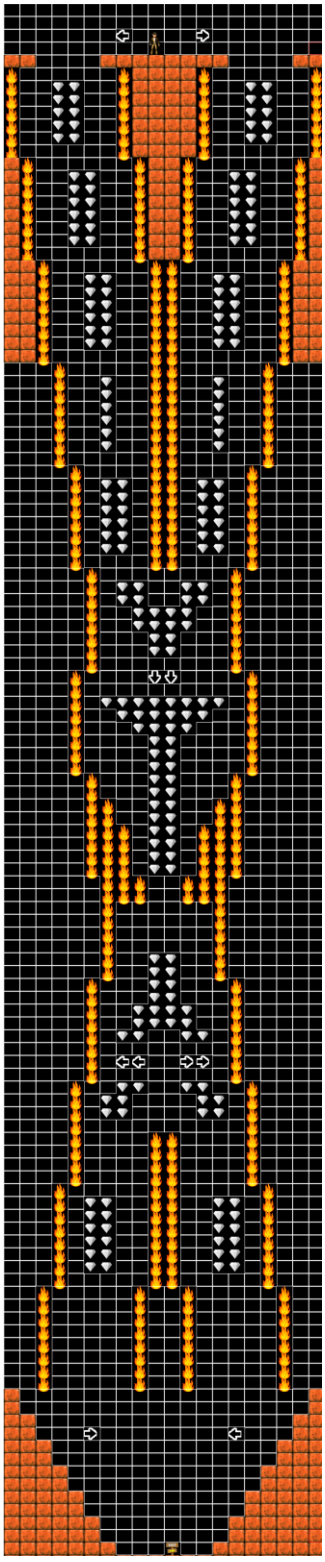


Figur 10 Decisions 1

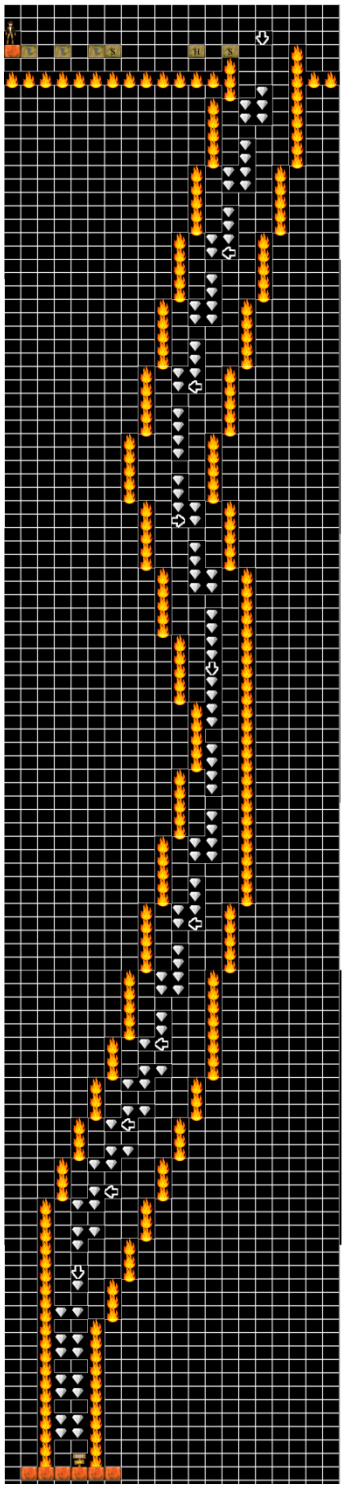


Figur 11 Decisions 2

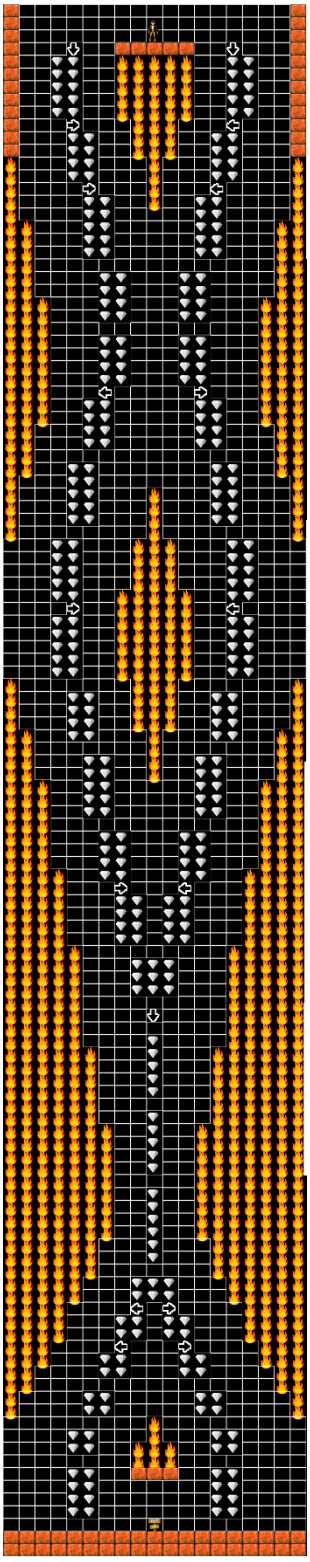
5. Nivåer med fall



Figur 12 Falling 1

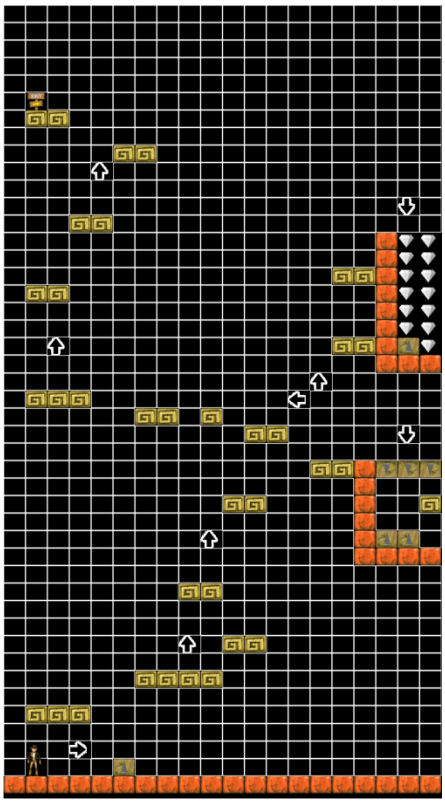


Figur 13 Falling 2

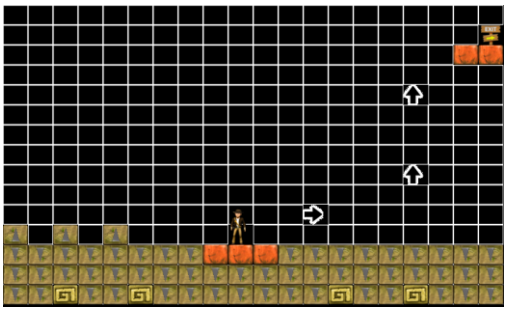


Figur 14 Falling 3

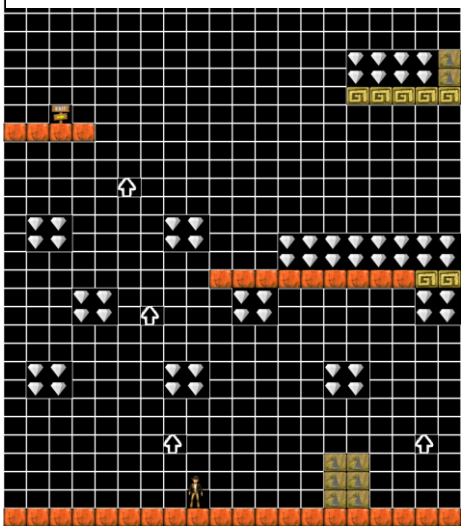
6. Nivåer med tongivande placerbara block



Figur 15 Influential Placeable Block 1

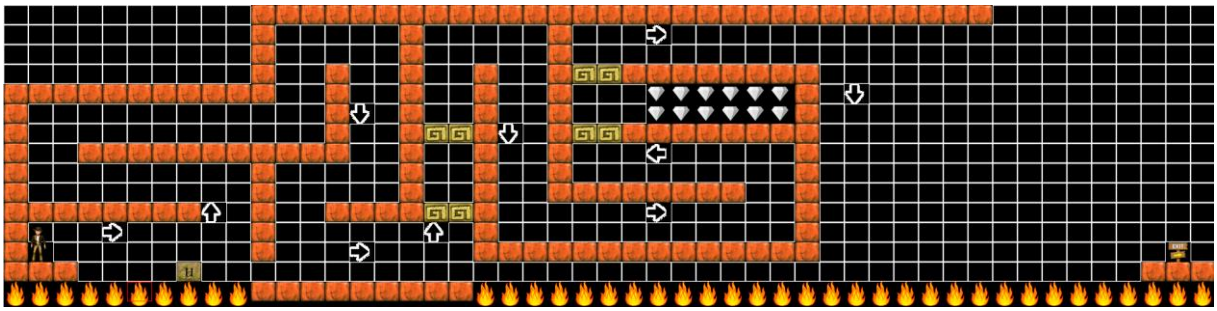


Figur 16 Influential Placeable Block 2

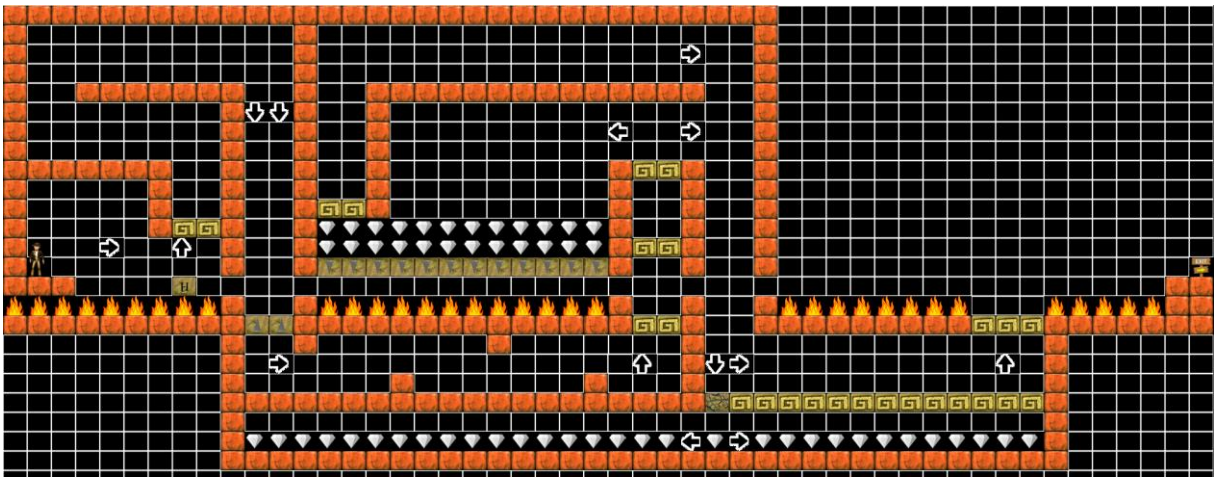


Figur 17 Influential Placeable Block 3

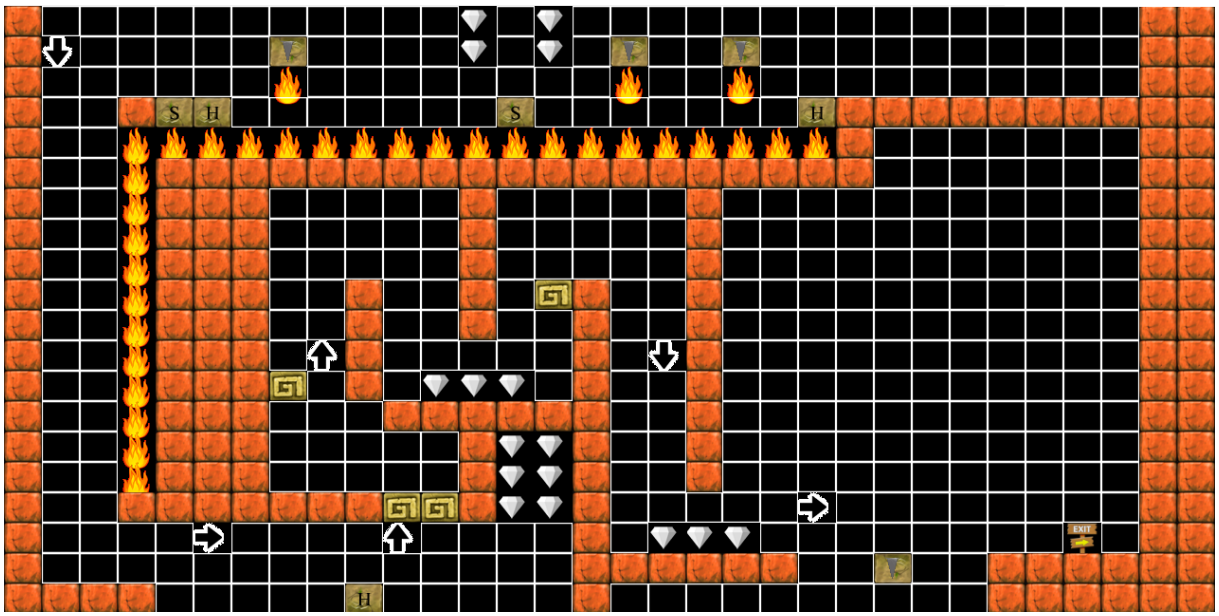
7. Horisontella nivåer som kräver rörliga horisontella



Figur 18 Forced Moving Blocks 1

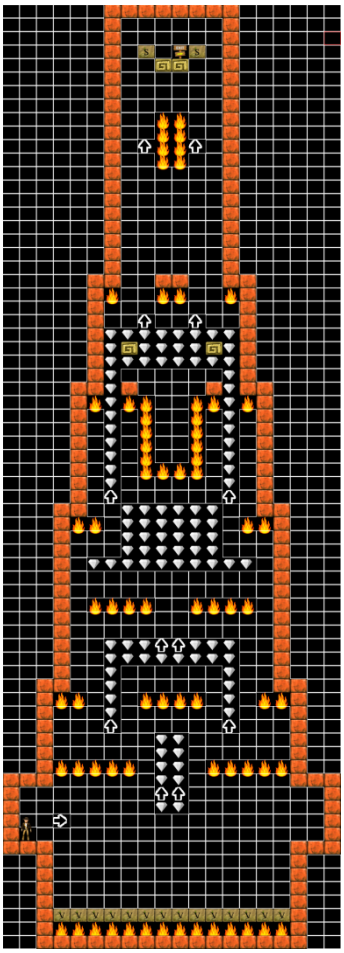


Figur 19 Forced Moving Blocks 2

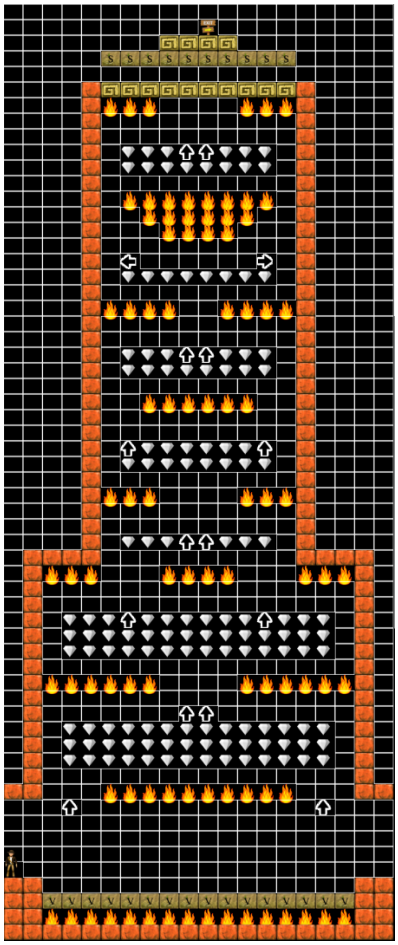


Figur 20 Forced Moving Blocks 3

8. Nivåer med vertikalt rörligt golv

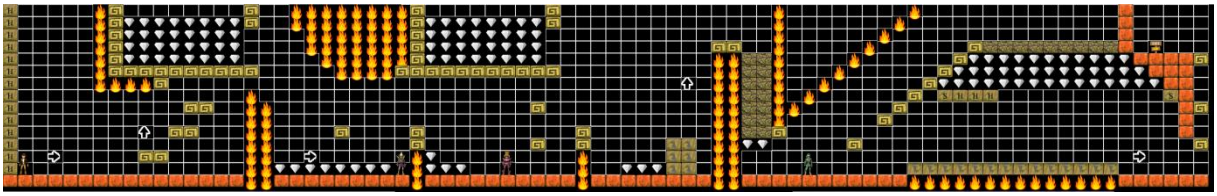


Figur 21 Moving Floor 1

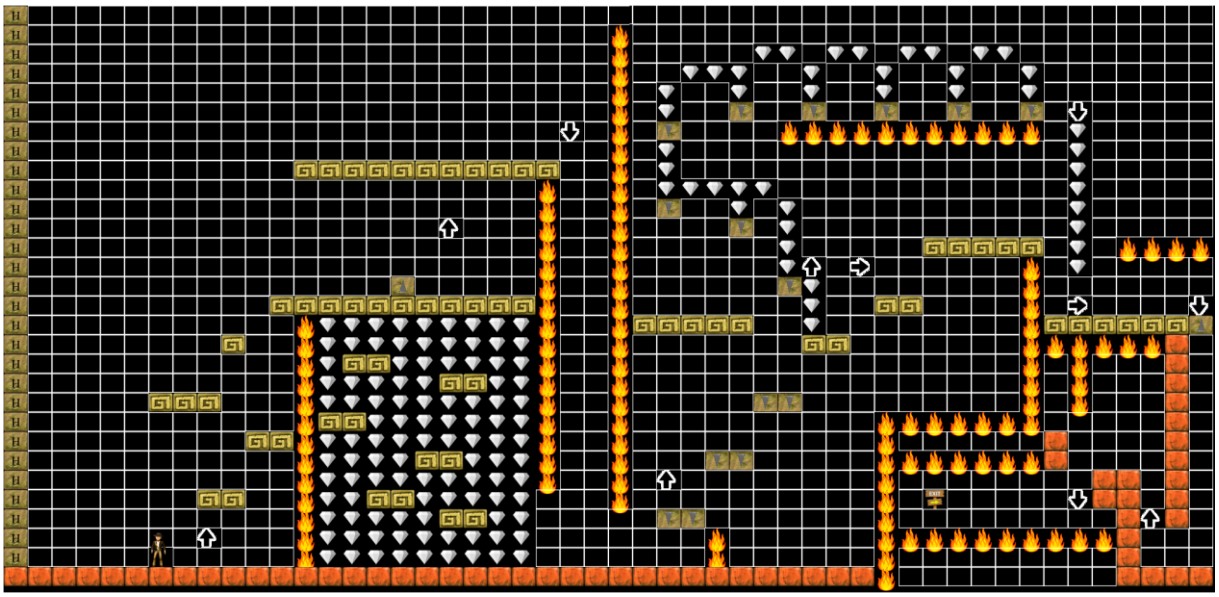


Figur 22 Moving Floor 2

9. Nivåer med horisontell rörlig vägg

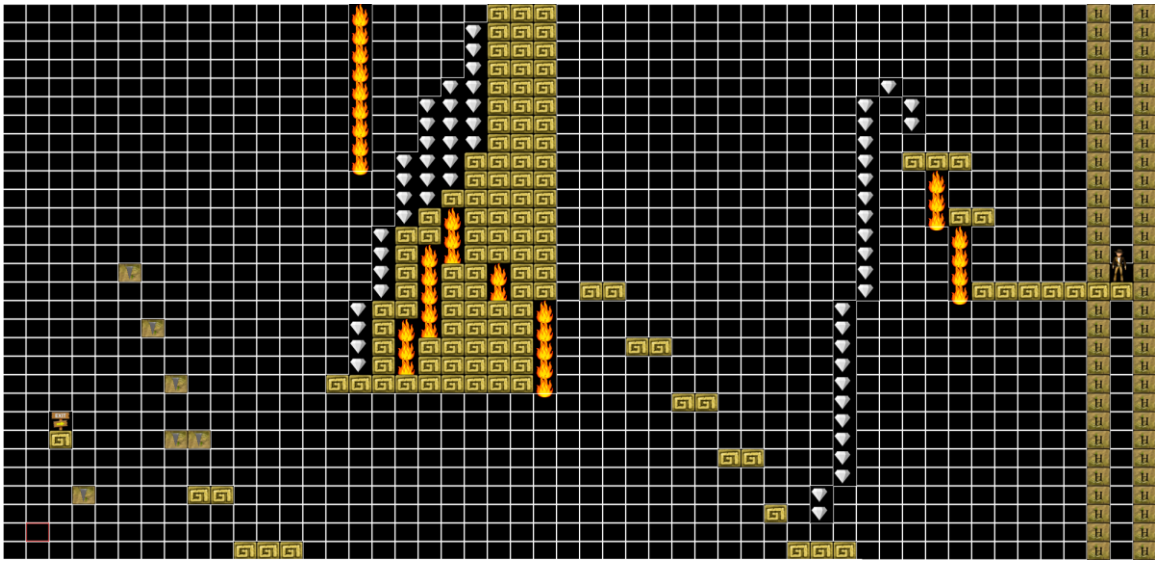


Figur 53 Moving Wall 1

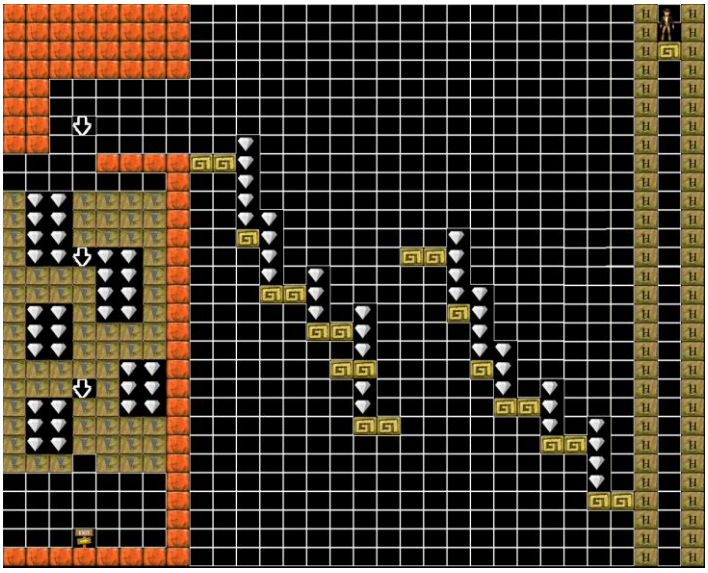


Figur 24 Moving Wall 2

10. Nivåer med två horisontella väggar

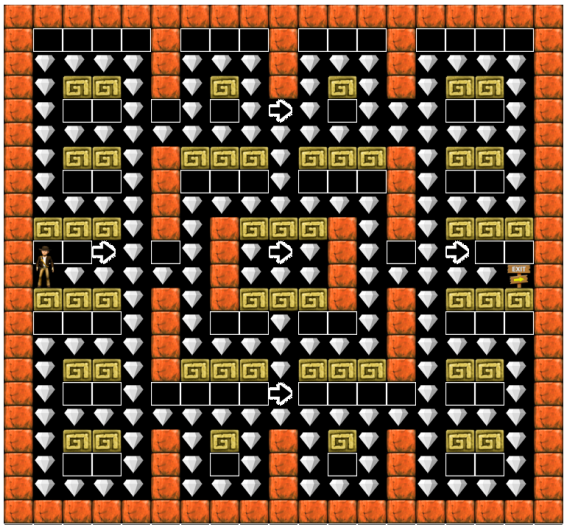


Figur 25 Pressed Together 1

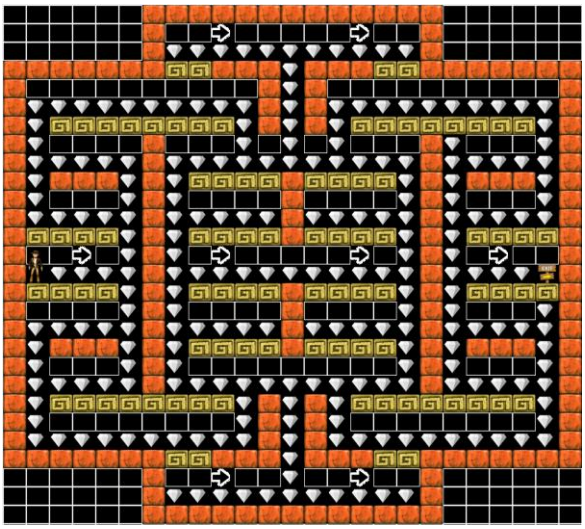


Figur 26 Pressed Together 2

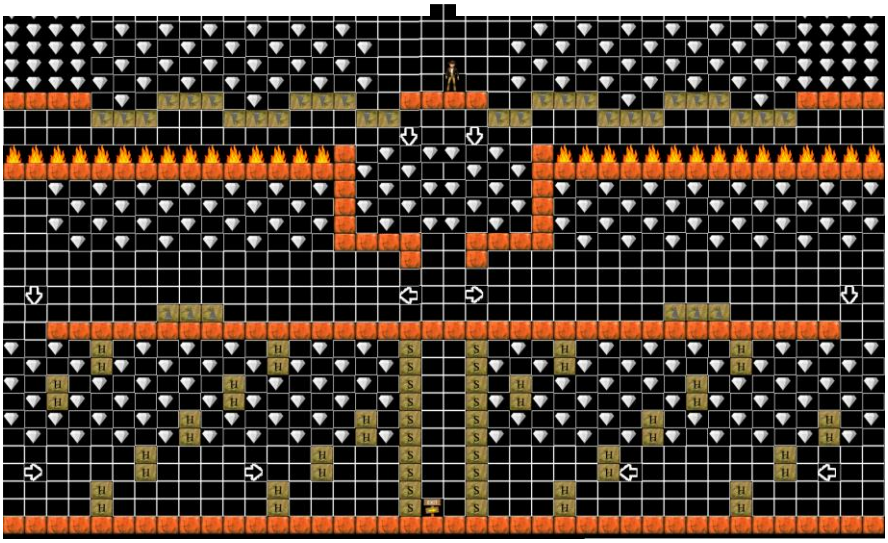
11. Nivåer med mycket gems



Figur 27 Gems 1

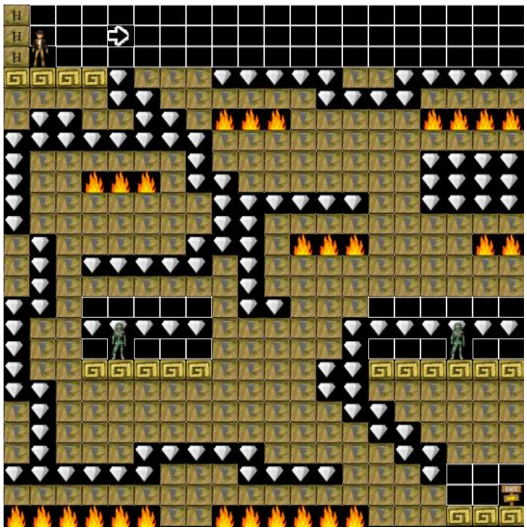


Figur 28 Gems 2

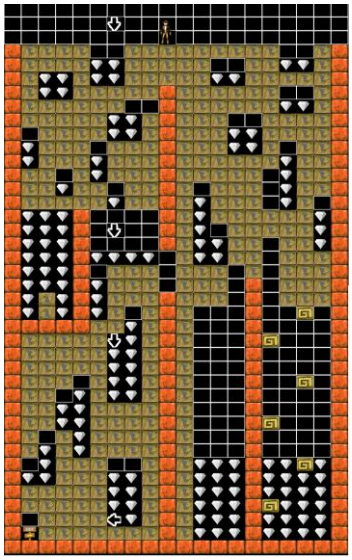


Figur 29 Gems 3

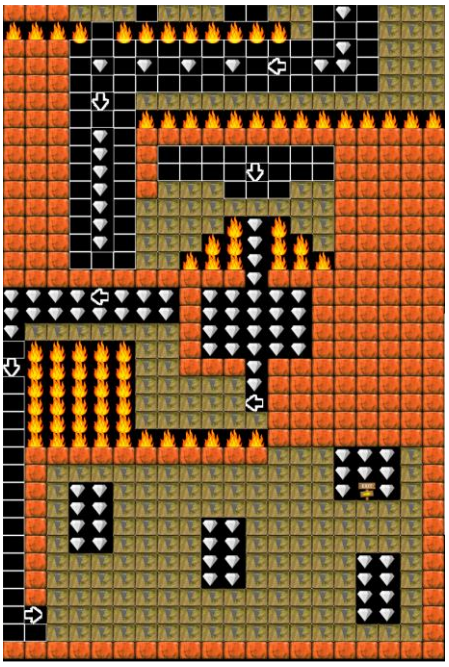
12. Nivåer med övervägande fallande block



Figur 30 Falling Blocks 1

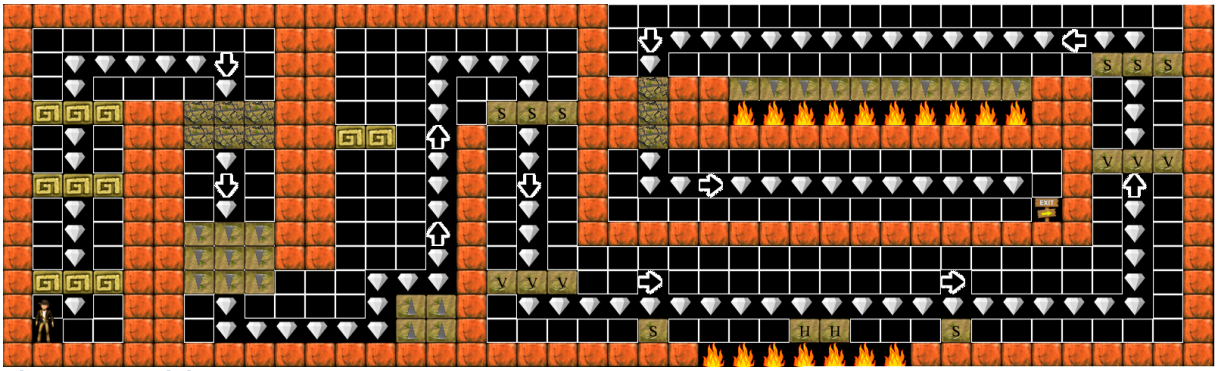


Figur 31 Falling Blocks 2

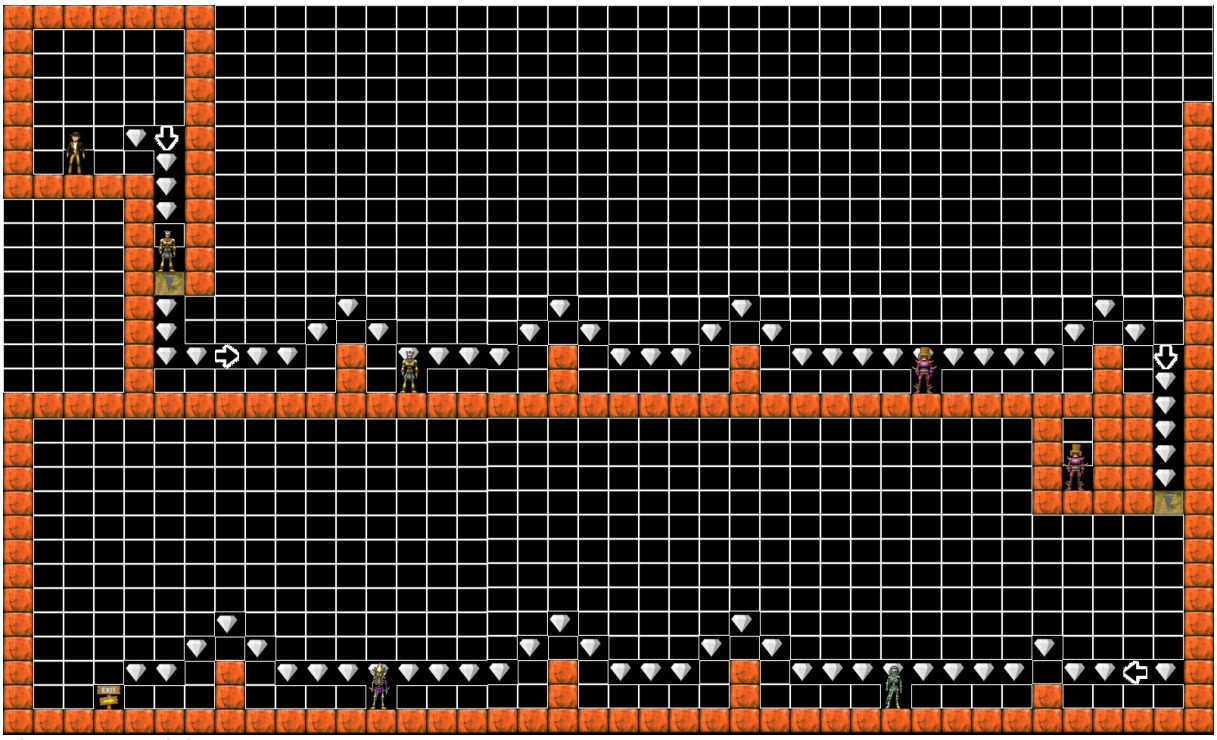


Figur 32 Falling Blocks 3

13. Tutorials



Figur 33 Tutorial 1



Figur 34 Tutorial 2

Appendix 3 – Nivåer och betyg

Average of Rating	Column Labels																							
Row Labels	u00	u01	u02	u03	u04	u05	u06	u07	u08	u09	u10	u11	u12	u13	u14	u15	u16	u17	u18	u19	u20	u21	u22	u23
Classic Horizontal 1	3	1			2		2	4	1	4	5	3	5	3	2	3	2	2		2	1	4	3	3
Classic Horizontal 2	3	1			2		5		1	4	5	4	5	4	4	1	3	1		3	3	4	3	3
Classic Horizontal 3	3	4			2	4	3		4			4	1		4	4	4	2		3	4	4	3	5
Classical Vertical 1	3	3	3		3		3	5	1	4		4	1	3	4	1	3	1		2	3	3	3	2
Classical Vertical 2	3	5	5		3		2		5		3	4	5		5	3	5	5		2	2	5	4	
Classical Vertical 3	2	2	4		2		5		2		3	4	5		2	3	4	3	3	3	3	3	5	2
Decisions 1	2	1			3	5	1		2		4	4	1	4	5	3	5	5		3	3	1	4	
Decisions 2	2	2	5		3		5	3	3		4	4	5		4	1	3	4		1	1	4	4	2
Falling 1	2	4	2		1	5	4	4	3			3	1	4	3	5	3	5	5	3	4	3	5	5
Falling 2	3	5	3		4		3	5				3	5	4	1	3	3	3		2	3	2	5	
Falling 3	3	4			1		4		1			2	1	3	3	3	2	5		3		3	3	
Falling Blocks 1	3	2			3	3		4	5	4		3	1	4	5	5	3	5		5	3	1	4	2
Falling Blocks 2	3	5			1	1	3		5	4	3		1	5	4	4	3	5		5	4	3	4	
Falling Blocks 3	3	2	5		4			3	4	5	5	4	5		4	4	4	5		5	4	5	5	4
Forced Moving Blocks 1	3	1			2		5	3	4			4	5		3	3	5	2	5	4	2	4	3	1
Forced Moving Blocks 2	3	4			3	4	5		5		5	4	5	1	4	3	4	5	2	4	4	5	4	5
Forced Moving Blocks 3	3	1			2		4	2		3	5	3	1		4	1	2	5		4	5	4	4	
Gems 1	3	3			2		3	3		4		4	5		5	3	2	5		2	2	1	2	4
Gems 2	3	4	4		3				2	4	3	4	1		3	2	4	5	3	4	3		4	
Gems 3	2	3	2		4	4	5		5		4	5	5	4	5	3	3	5	1	4	3	4	3	2
Influental Moving Blocks 1	3	2			3		4		4	4		5	5	3	4	2	2	3		4	3		5	
Influental Moving Blocks 2	3	5	5		2	4	3		2	4		3	5	2	4	3	3	1	4	1	2	4	2	5
Influental Moving Blocks 3	3	4	3	3	2		4		1			3	5	4	4	5	2	3		5	2	2	3	3
Influental Placeable Blocks 1	3	4			3		5	3	4	4		4	5	5	4	4	2	4		3	2	4	2	5
Influental Placeable Blocks 2	3	4			4		5		4	3	4	2	5	5	3	3	5	5		4	2	3	5	5
Influental Placeable Blocks 3	2	2			4	3	5	4	4	4	4	4	5		2	3	4	5	3	4	3	4	2	5
Moving Floor 1	3	2	4		2	4	4			4		4	5	5	4	4	2	5		2	3	4	4	2
Moving Floor 2	3	5	4		3		3		2		5	3	5	4	4	3	5	4		3	3	3	5	3
Moving Wall 1	3	5	4		2	4	4	4	5	4		5	5		4	1	5	5		5	3	5	3	1
Moving Wall 2	4	5	5		5		4	3	4	4	5	4	3	5	5	3	4	5		4	3	4	5	
Pressed together 1	4	3	4		4	4	5	4	5	4		3	5		5	3	5	5		5	4	5	4	3
Pressed together 2	3	1	2		2		2	4	5			3	5	4	5	4	4	4		3	3	2	5	
Grand Total	**2,88**	**3,09**	**3,76**	**3**	**2,69**	**3,75**	**3,79**	**3,63**	**3,32**	**3,94**	**4,19**	**3,65**	**3,81**	**3,8**	**3,81**	**3**	**3,44**	**3,97**	**3,25**	**3,34**	**2,9**	**3,43**	**3,75**	**3,27**

Average of Rating																							
Row Labels	u24	u25	u26	u27	u28	u29	u30	u31	u32	u33	u34	u35	u36	u37	u38	u39	u40	u41	u42	u43	u44	Grand Total
Classic Horizontal 1	3	4		4	2	2	5	2	3	3	2	3	4	4	5	3	4		5	3	2	3,03
Classic Horizontal 2		2		3	1	3	5	2	5	3	3	5		4	1	1	4	3	1	4	1	2,97
Classic Horizontal 3	4	2	5	4	4	2	4	2	3	2	4	3	3	3	5	2	3	3	5	5	4	3,42
Classical Vertical 1	2	2		5	1	4	4	1	4	3	3	5	5	5	2	4	3		2	5	4	3,05
Classical Vertical 2		4	5	5	4		5	4	3	5	4	5	1	5		4	4	4	1	5		3,94
Classical Vertical 3		2		4	3	2	5	3	3	4	4	3	5	4	2	2	4	4	3	4	5	3,32
Decisions 1		2		5	4	3	5	4	5	3	4	2	2	5	4	5	4	3	4	4	4	3,46
Decisions 2	4	2	4	5	3	1	3	4	1	4	3	3	4	3	1	1	4	1	1	3	5	3
Falling 1	4	3		5	2	4	5	2	1	4	2	2	1	5	2	5	3		4	5	4	3,43
Falling 2		4		5	2		5	2	4	3	2	4	1	5	3	4	4	4	4	4		3,44
Falling 3		3		1	3	4	5	2	2	4		1	4	5	5	5	4		4	4		3,13
Falling Blocks 1	4	3	4	5	1	3	5	4	2	3	2	4	1	5	5	5	4		4	4	5	3,54
Falling Blocks 2		1	4	4	2	2	4	4	5	4		3	5	2	4	3	4	3	1	4	4	3,41
Falling Blocks 3	3	4	4	5	4	2	5	2	4	4	4	1	5	4	2	2	4	3	3	4	4	3,83
Forced Moving Blocks 1	4	3		5	4	1	5	1	4	5	4	3		5	1	2	3	2	1	4	2	3,19
Forced Moving Blocks 2	3	4	5	5	4	2	5	3	4	4	3	4		5	2	5	4	2	5	5	4	3,93
Forced Moving Blocks 3	3	3		5	3	2	5	4	1	2	4	3	4	5	5	2	4		5	2		3,29
Gems 1	3	2		5	4	4	4	1	4	4	1	4		4	5	4	4		4	5		3,38
Gems 2	5	2	5	2	1		5	1	4	4	3	2	4	1		3	4	3	3	5	5	3,28
Gems 3	4	3		5	1		3	4	4	2	4	3		4	3	4	4	3	5	5	3	3,59
Influental Moving Blocks 1	2	3	4	5	2	4	5	3	3	4	4	3		4		1	4		5	2	3	3,44
Influental Moving Blocks 2	1	3		5	5	2	5	1	3	3	4	3	2	5	3	4	4		3	3	2	3,2
Influental Moving Blocks 3	1	4	4	5	5	3	3		1	2	2	2	1	5	4	5	3	3	4	4	3	3,21
Influental Placeable Blocks 1	5	3	4	4	1	3	5	2	2	4	4	5	2	5	3	4	3	4	1	4	4	3,55
Influental Placeable Blocks 2	5	3	5	5	1	2	4	1	3	3	3	2		5	2	5	4	2	2	5	5	3,62
Influental Placeable Blocks 3	4	3		5	1	2	5	4	2	2	4	5	2	5	4	4	4		4	5	4	3,63
Moving Floor 1	2	3	4	5	3	2	5	3	2	2	2	4	1	3	5	4	4		2	4		3,34
Moving Floor 2		3		5	4	3	5	3	1	3	4	5	2	5	4	4	4		4	5	4	3,73
Moving Wall 1	5	3			5	4	3	4	5	5		5		5		5	4		5	5	4	4,11
Moving Wall 2	4	4		5	3	3	4	3	5	5	5	5		3	3	4	5	2	5	5	4	4,13
Pressed together 1	5	3	4	5	3	3	5	3	5	3		4	4	1	3	4	5	5	5	5	5	4,1
Pressed together 2	3	2	5	5	4	3	5	2	1	1	3	4		4	5	3	4	3	5	5	4	3,47
Grand Total	**3,46**	**2,88**	**4,4**	**4,55**	**2,81**	**2,68**	**4,56**	**2,61**	**3,09**	**3,34**	**3,25**	**3,44**	**2,86**	**4,16**	**3,32**	**3,53**	**3,88**	**3**	**3,44**	**4,25**	**3,77**	**3,5**

Högskolan i Borås är en modern högskola mitt i city. Vi bedriver utbildningar inom ekonomi och informatik, biblioteks- och informationsvetenskap, mode och textil, beteendevetenskap och lärarutbildning, teknik samt vårdvetenskap.

På **institutionen Handels- och IT-högskolan (HIT)** har vi tagit fasta på studenternas framtida behov. Därför har vi skapat utbildningar där anställningsbarhet är ett nyckelord. Ämnesintegration, helhet och sammanhang är andra viktiga begrepp. På institutionen råder en närhet, såväl mellan studenter och lärare som mellan företag och utbildning.

Våra **ekonomiutbildningar** ger studenterna möjlighet att lära sig mer om olika företag och förvaltningar och hur styrning och organisering av dessa verksamheter sker. De får även lära sig om samhällsutveckling och om organisationers anpassning till omvärlden. De får möjlighet att förbättra sin förmåga att analysera, utveckla och styra verksamheter, oavsett om de vill ägna sig åt revision, administration eller marknadsföring. Bland våra **IT-utbildningar** finns alltid något för dem som vill designa framtidens IT-baserade kommunikationslösningar, som vill analysera behov av och krav på organisationers information för att designa deras innehållsstrukturer, bedriva integrerad IT- och affärsutveckling, utveckla sin förmåga att analysera och designa verksamheter eller inrikta sig mot programmering och utveckling för god IT-användning i företag och organisationer.

Forskningsverksamheten vid institutionen är såväl professions- som design- och utvecklingsinriktad. Den övergripande forskningsprofilen för institutionen är handels- och tjänsteutveckling i vilken kunskaper och kompetenser inom såväl informatik som företagsekonomi utgör viktiga grundstenar. Forskningen är välrenommerad och fokuserar på inriktningarna affärsdesign och Co-design. Forskningen är också professionsorienterad, vilket bland annat tar sig uttryck i att forskningen i många fall bedrivs på aktionsforskningsbaserade grunder med företag och offentliga organisationer på lokal, nationell och internationell arena. Forskningens design och professionsinriktning manifesteras också i InnovationLab, som är institutionens och Högskolans enhet för forskningsstödjande systemutveckling.

HÖGSKOLAN I BORÅS

VETENSKAP FÖR PROFESSION

BESÖKSADRESS: JÄRNVÄGSGATAN 5 · POSTADRESS: ALLÉGATAN 1, 501 90 BORÅS
TFN: 033-435 40 00 · E-POST: INST.HIT@HB.SE · WEBB: WWW.HB.SE/HIT